WSQ COURSE CODE · TGS-2026061312

Claude Certified Architect Foundation

Trainer: Dr Alfred Ang · Version 2.3 · 6 Nov 2026

Tertiary Infotech Academy Pte Ltd
UEN: 201200696W

www.tertiarycourses.com.sg | enquiry@tertiaryinfotech.com | +65 6100 0613

© Tertiary Infotech Academy Pte Ltd

Digital Attendance (Mandatory)

Three scans a day keep your funding eligibility intact.

1. Trainer displays the SSG QR code
2. Scan with your phone camera
3. Submit your attendance
4. Repeat: AM, PM and Assessment

75%

minimum attendance for assessment and funding

Mandatory for WSQ-funded courses

Take the AM, PM and Assessment digital attendance. Miss a scan, and tell your trainer the same day.

© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

About the Trainer



Your Trainer

General Trainer template: to be completed by the trainer

NAME

TITLE / DESIGNATION

QUALIFICATIONS

AREAS OF EXPERTISE

TRAINING & INDUSTRY EXPERIENCE

CONTACT

© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

YOUR TRAINER

About the Trainer



Dr Alfred Ang

Lead Trainer, Tertiary Infotech Academy Pte Ltd

FOCUS

Practical agent architectures, automation and evidence-driven engineering

IN THE CLASSROOM

Connects each mechanism to workplace constraints, failure modes and acceptance evidence

WHAT YOU LEAVE WITH

Systems that are bounded, observable, resumable and safe to review

© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

ICE-BREAKER

Let's Know Each Other

Sixty seconds each. Your answers shape which labs we go deep on.

Who you are

Your name, organisation and role

What you've built

Anything with the Claude API, Claude Code or MCP so far

What you want

The agent or workflow you want running after this course

The exam

Sitting CCAR-F? When?

© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

HOUSEKEEPING

Ground Rules

Six habits that make two days of live agent runs work.

Phone on silent

Agents may run loud. Phones should not.

Ask anything

No question is too small

Agree to disagree

Mutual respect, always

One conversation

One person talks at a time

Be punctual

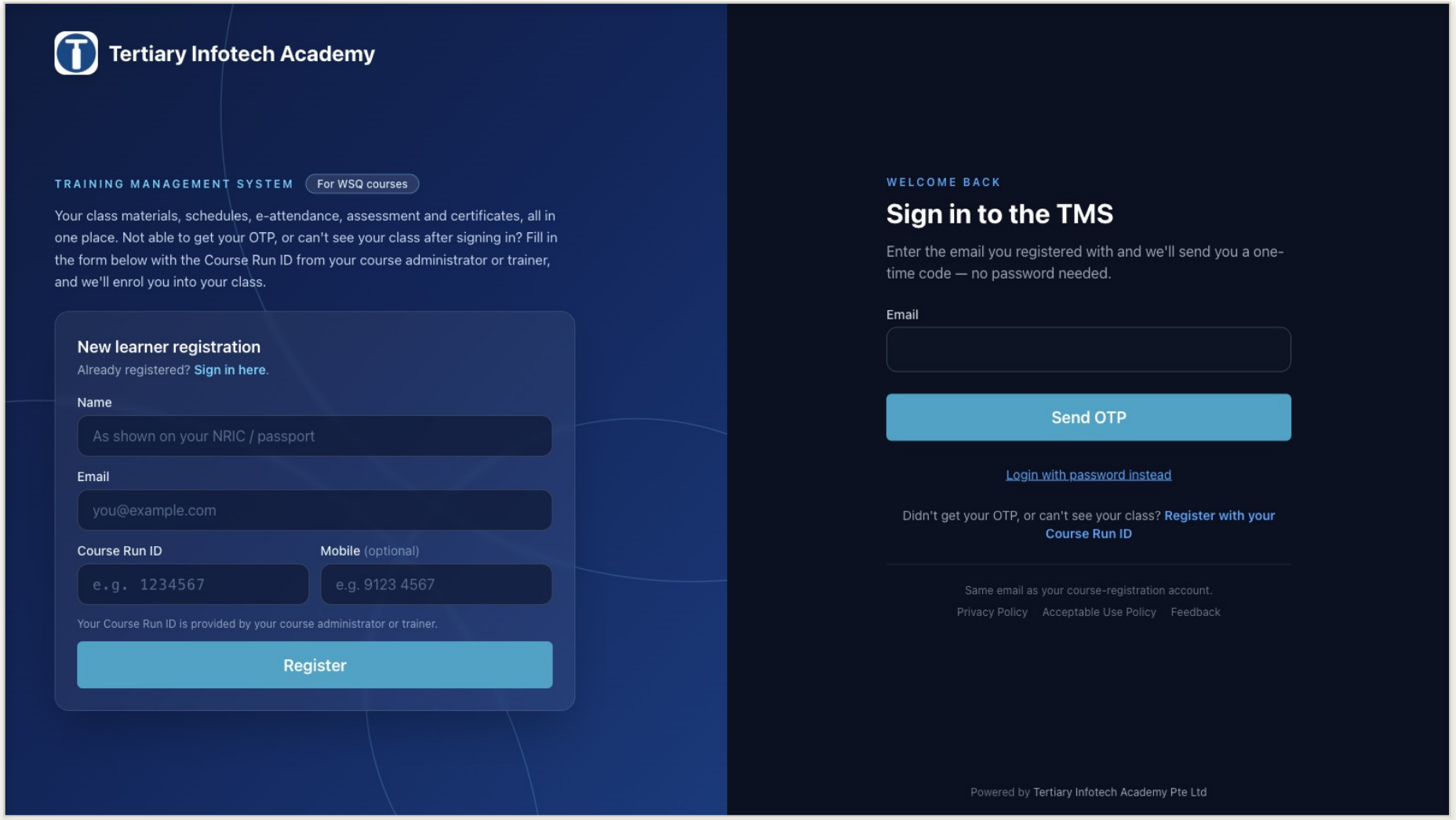
Return from breaks on time

Attend 75%+

Required for assessment and funding

© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Login to LMS and Download Course Material



© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Lesson Plan · Day 1

Each topic alternates concept and lab. Nothing waits until the end.

Time	Session	Labs in class
09:00 to 09:30	Welcome, exam blueprint, six simulators	Setup check
09:30 to 11:30	Topic 1: Agentic Architecture & Orchestration	Labs 1, 3, 5, 6, 9, 10
11:30 to 11:45	Break	
11:45 to 13:00	Topic 1, continued	Labs 12, 15, 16
13:00 to 14:00	Lunch	
14:00 to 15:30	Topic 2: Tool Design & MCP Integration	Labs 18, 19, 21, 23, 25
15:30 to 15:45	Break	
15:45 to 18:00	Topic 3: Claude Code Configuration & Workflows	Labs 27, 29, 30, 31, 33, 36

© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Lesson Plan · Day 2

Assessment sits at the end, with a checkpoint before it.

Time	Session	Labs in class
09:00 to 11:15	Topic 4: Prompt Engineering & Structured Output	Labs 38, 40, 43, 44, 45, 48, 50
11:15 to 12:15	Topic 5: Context Management & Reliability	Labs 51, 54
12:15 to 12:30	Break	
12:30 to 13:00	Topic 5, continued	Lab 55
13:00 to 14:00	Lunch	
14:00 to 15:00	Topic 5, continued	Labs 56, 60, 61
15:00 to 15:45	Recap and assessment briefing	
15:45 to 16:00	Break	
16:00 to 17:00	Written Assessment	
17:00 to 18:00	Case Study	

© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

LEARNING OUTCOMES

What you will be able to engineer

Five outcomes, one for each exam domain.

LO1

Design bounded agentic loops and multi-agent orchestration with explicit handoffs, state and stop conditions.

LO2

Engineer reliable tool and MCP integrations with precise schemas, structured errors and least-privilege tool distribution.

LO3

Configure Claude Code projects with scoped instructions, reusable skills, rules, plan mode and CI/CD workflows.

LO4

Produce validated structured outputs using clear criteria, few-shot examples, batching and review loops.

LO5

Manage context, uncertainty, provenance and human escalation in large, long-running AI systems.

How you prove it

61 labs on six simulators, then the Written Assessment and the Case Study.

© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Briefing for Assessment

Six rules for the two papers on Day 2.

Phones away

Phones and other materials go under the table or on the floor

No photos

No photos or recording of assessment scripts

Silence

No discussion during the assessment

Pen

Black or blue pen for hard-copy papers

No correction tape

No liquid paper either

Time up

Scripts are collected when time is up

© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Assessment

Two papers, one hour each, open book.

WA

Written Assessment (SAQ): 10 questions, 60 minutes

CS

Case Study: 5 tasks, one lab screenshot each, 60 minutes

LMS

Submit your completed answers there

Before you start

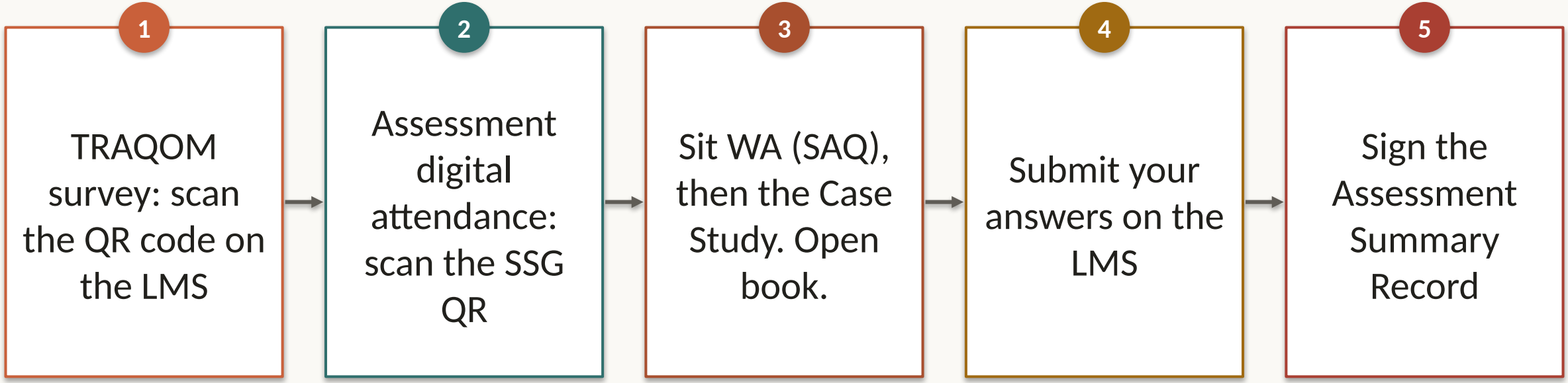
Open book: slides, Learner Guide and approved materials only.
Complete the TRAQOM survey, then the Assessment digital attendance.
Each Case Study task needs one screenshot from a lab you run yourself.
An appeal process is available if you need one.

© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

ON ASSESSMENT DAY

Assessment Flow

Five steps, in this order, every time.



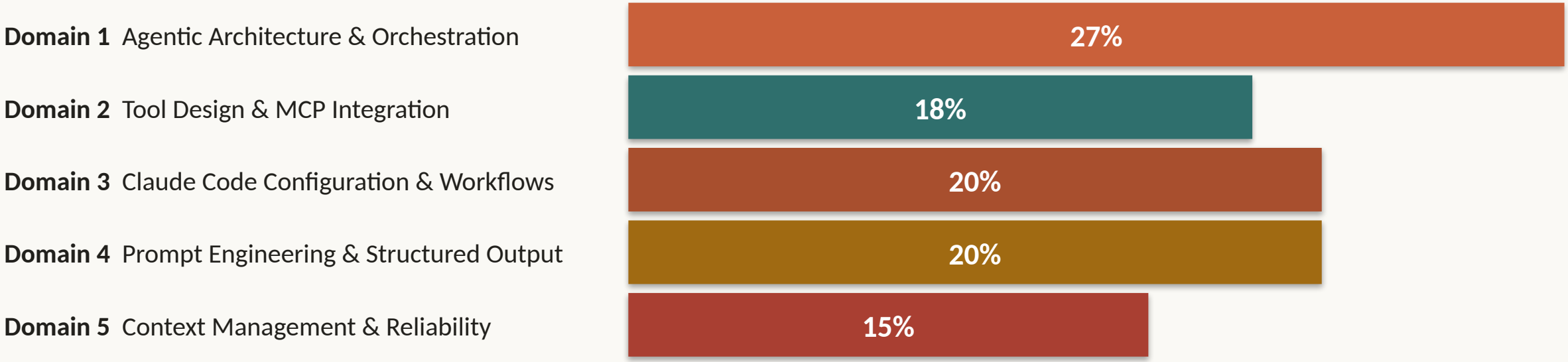
TRAQOM QR code and answer submission both sit on the LMS: lms-tms.tertiaryinfotech.com

© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

60 items, 4 scenarios drawn from a bank of 6

You can't guess which four. So you practise on all six.

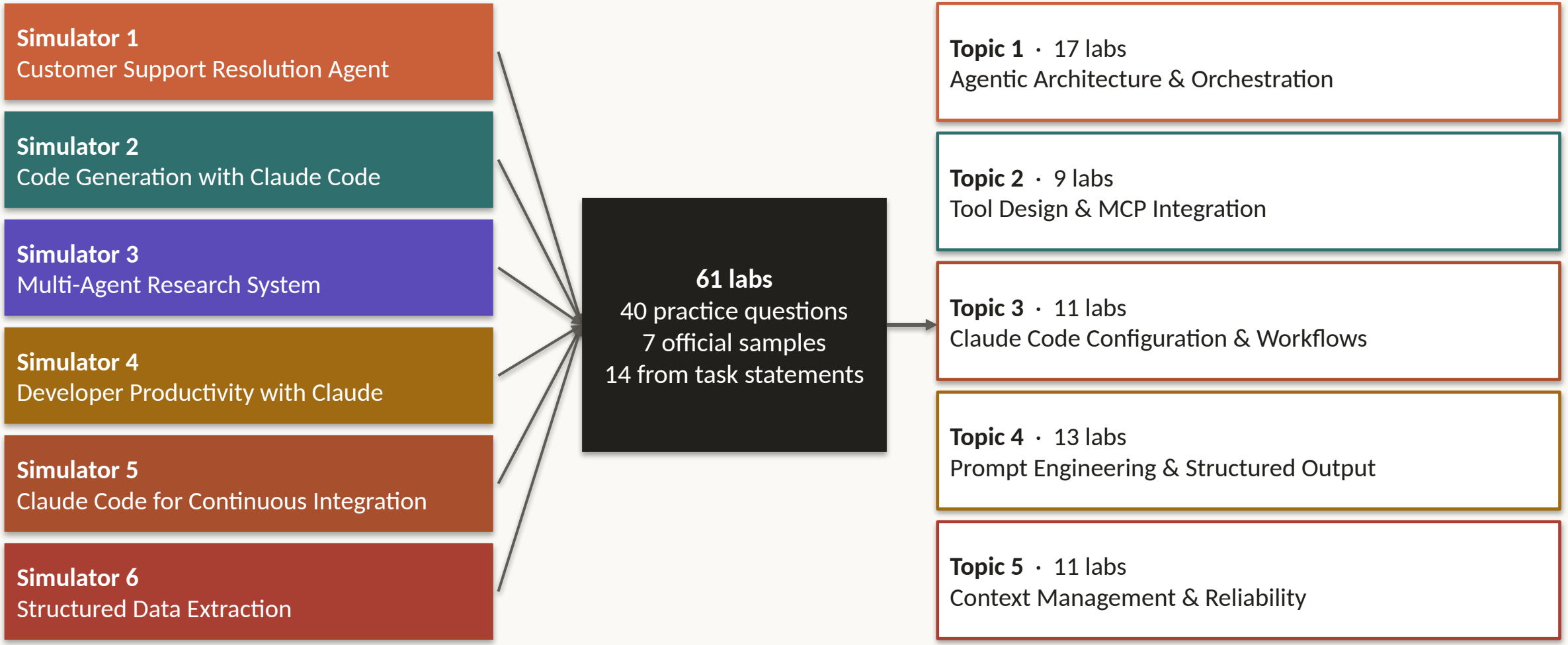
60	120	720	$125	12
scenario-based items	minutes, proctored	scaled pass mark, on 100 to 1,000	exam fee in USD	months of validity



© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Six simulators, 61 labs, one exam situation at a time

Python and the Claude Agent SDK, in your browser. Each lab is one exam question: answer it, then simulate the solution.

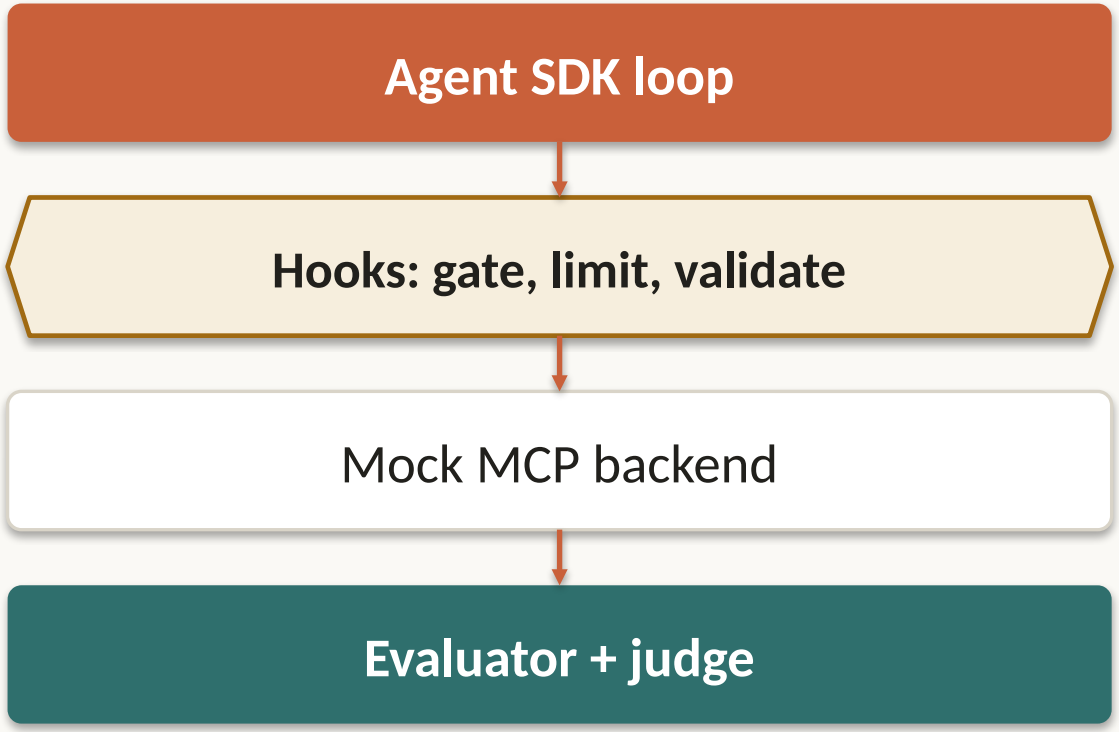


© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Your agent has refund powers. Prove it knows when not to use them.

Exam guide, word for word

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (get_customer, lookup_order, process_refund, escalate_to_human). Your target is 80%+ first-contact resolution while knowing when to escalate.



Primary domains

Agentic Architecture, Tool Design & MCP, Context & Reliability

11 labs run on this simulator

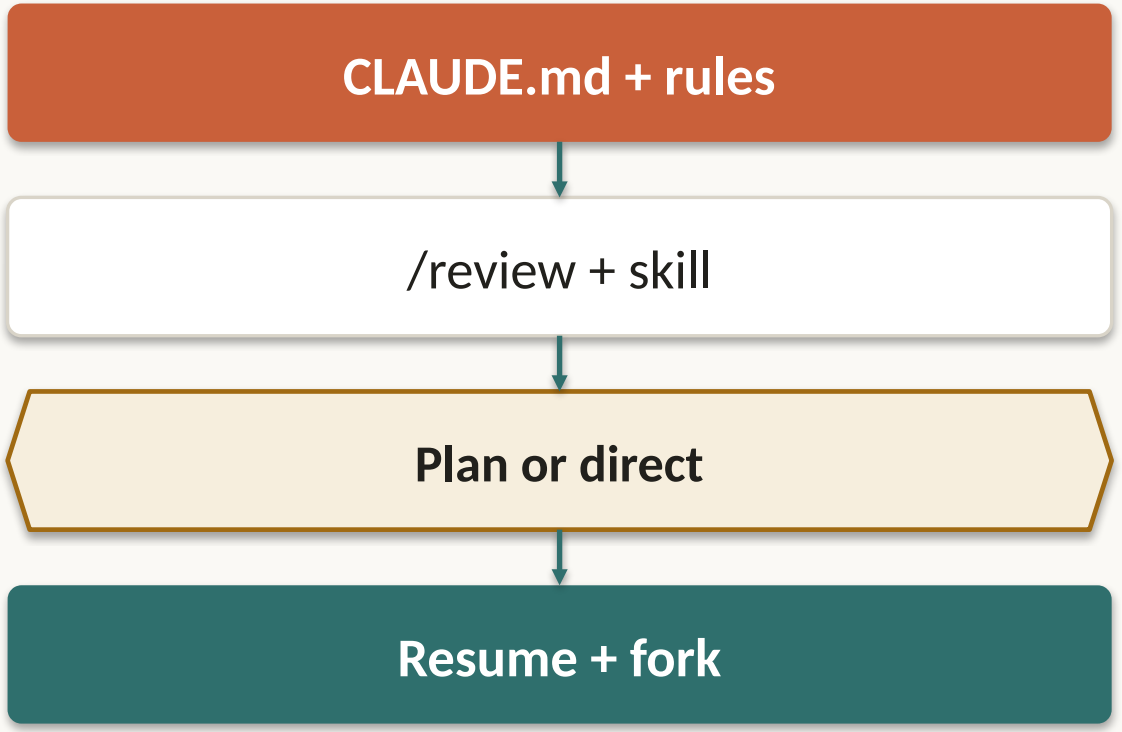
1, 9, 12, 19, 20, 29, 43, 45, 46, 51, 54

© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Claude Code, configured so the whole team gets the same teammate

Exam guide, word for word

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.



Primary domains

Claude Code Configuration, Context & Reliability

12 labs run on this simulator

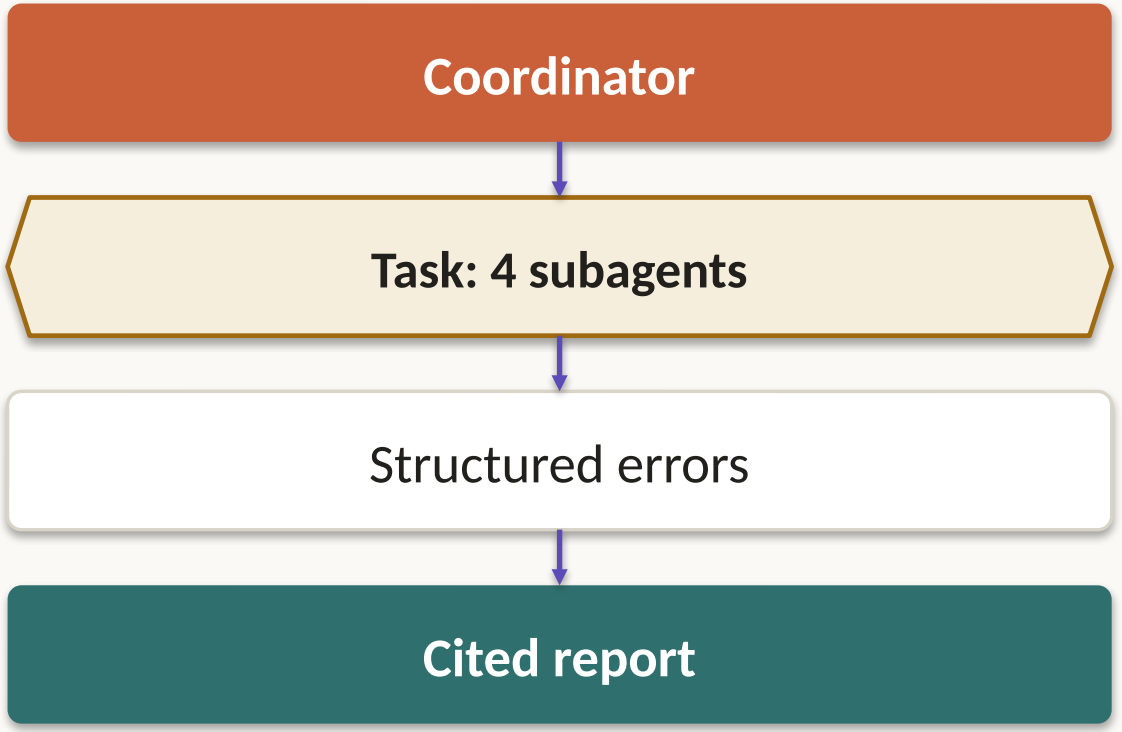
2, 5, 13, 16, 23, 27, 28, 31, 32, 33, 40, 59

© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

One coordinator, four specialists, zero invented citations

Exam guide, word for word

You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.



Primary domains

Agentic Architecture, Tool Design & MCP, Context & Reliability

13 labs run on this simulator

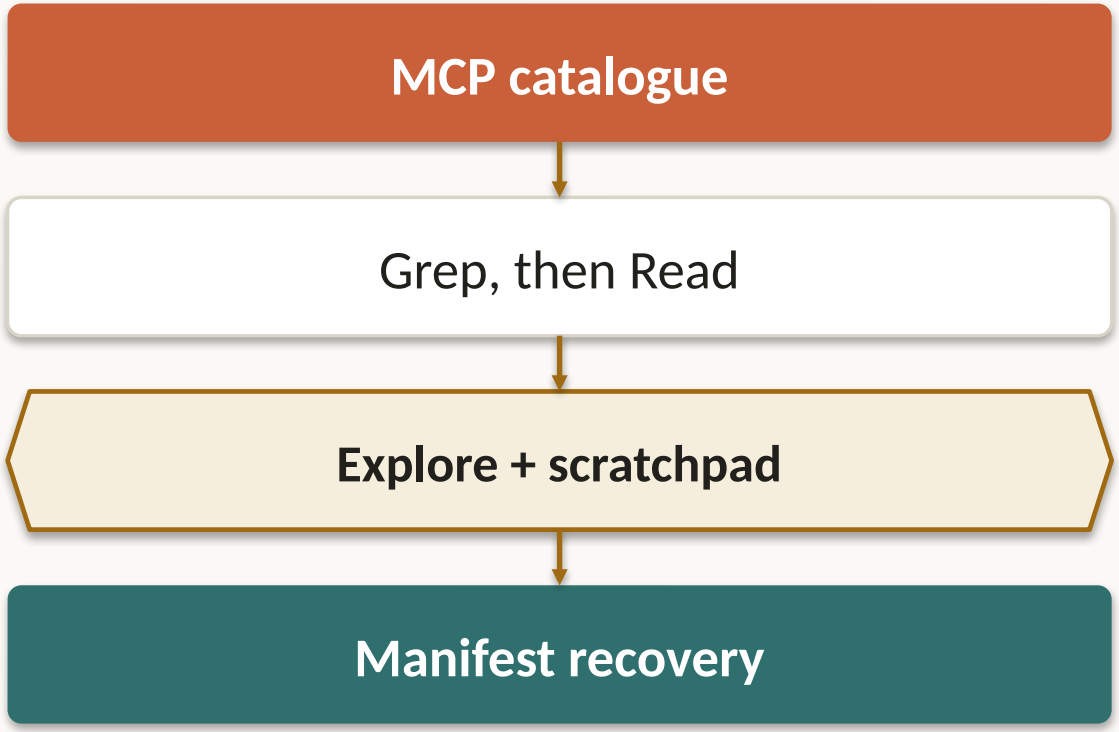
3, 7, 8, 22, 30, 35, 39, 41, 48, 49, 53, 55, 61

© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Find your way around a legacy repo without drowning in it

Exam guide, word for word

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.



Primary domains

Tool Design & MCP, Claude Code Configuration, Agentic Architecture

7 labs run on this simulator

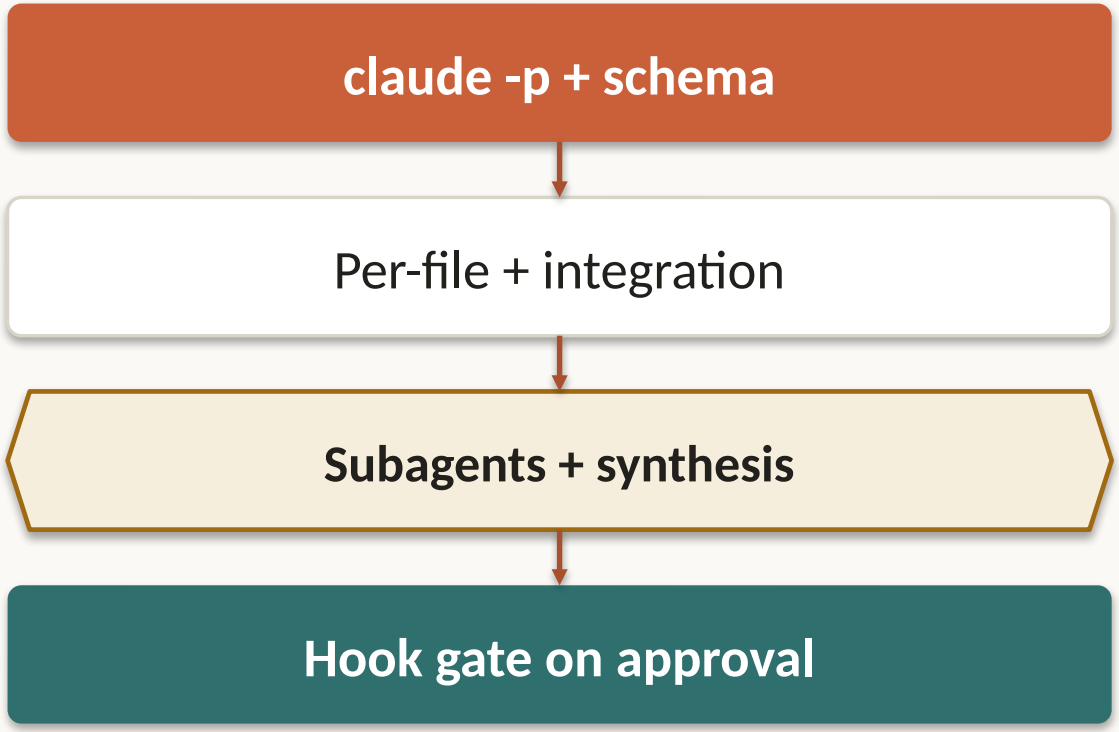
14, 17, 24, 25, 26, 56, 58

© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

A reviewer in the pipeline that developers don't learn to ignore

Exam guide, word for word

You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.



Primary domains

Claude Code Configuration, Prompt Engineering

9 labs run on this simulator

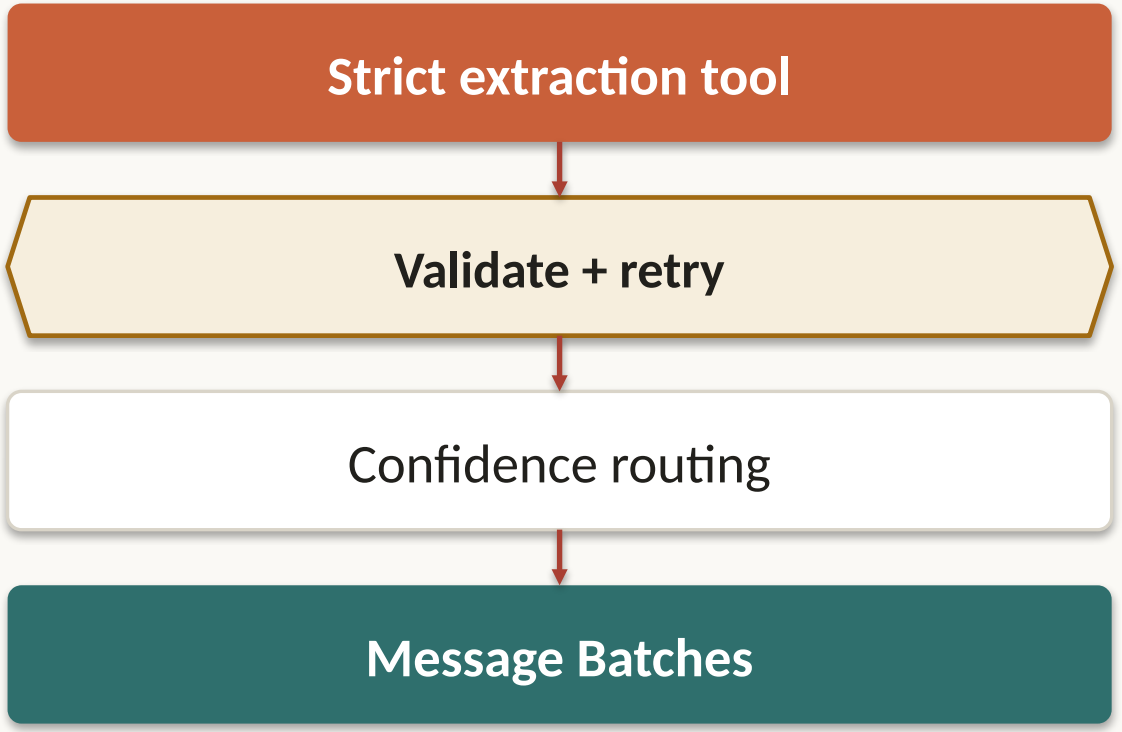
6, 10, 21, 34, 36, 37, 38, 50, 52

© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Messy documents in, trustworthy records out

Exam guide, word for word

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.



Primary domains

Prompt Engineering, Context & Reliability

9 labs run on this simulator

4, 11, 15, 18, 42, 44, 47, 57, 60

© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

SETUP

Ten minutes now saves every lab later

Everything runs on your laptop, against mock backends.

From the course folder

```
claude auth login

cd labs
uv sync
uv run serve.py
# the six simulators open in your browser
```

Your login does the work

All six run on the Claude Agent SDK, through your Claude login. No key file.

Python and uv

uv fetches Python and the SDK. One command starts all six.

Watch the spend

The page's Model menu picks `claude-sonnet-5-5` by default, `claude-haiku-5-5` to spend least. A full pass through every simulator costs roughly US$30 to 40 on Opus. Short on time or budget? Every lab page has a free Show the saved run button.

© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Commit to an answer before the simulator shows you

The exam tests judgement. So each lab makes you judge, then gives you evidence.



Where the questions come from

SkillCertPro's Part 1 practice exam, plus the exam guide's official samples. SkillCertPro is third-party and unofficial: practice material, not real exam items.

How to read a lab slide

Red: the tempting wrong design. Teal: the answer. Gold: what you should see. Run it on the simulator's page, with /lab N in Claude Code, or with the command shown. Speaker notes hold the full question and the reason each distractor fails.

© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

TOPIC 1 · EXAM DOMAIN 1 · 27% OF THE EXAM

Agentic Architecture & Orchestration

Loops, coordinators, hooks, sessions: how the agent decides and what stops it.



© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

TOPIC 1 MAP

7 task statements, 17 labs, 6 simulators

Concept first, then its labs, straight away.

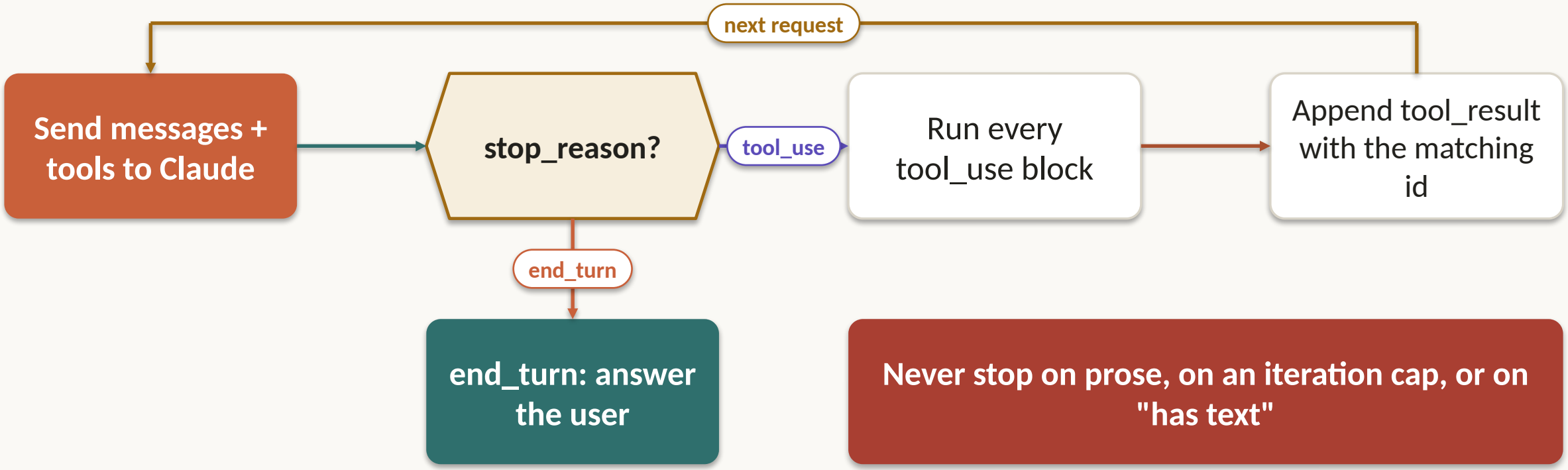
Task statement	Sim 1	Sim 2	Sim 3	Sim 4	Sim 5	Sim 6
1.1 Design and implement agentic loops for autonomous task execution	1	2				
1.2 Orchestrate multi-agent systems with coordinator-subagent patterns			3			4
1.3 Configure subagent invocation, context passing, and spawning		5	7,8		6	
1.4 Implement multi-step workflows with enforcement and handoff patterns	9				10	11
1.5 Apply Agent SDK hooks for tool call interception and data normalization	12	13		14		
1.6 Design task decomposition strategies for complex workflows						15
1.7 Manage session state, resumption, and forking		16		17		

© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

The loop runs on stop_reason, and nothing else

Parse the prose for "done" and your agent stops when it feels like stopping.

Claude picks the next tool from what it just learned. No hard-coded tree.



© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Ten lines hold the whole agent

Learn this loop and every SDK abstraction above it makes sense.

The whole control loop

```
while True:
    r = client.messages.create(
        model=MODEL, tools=TOOLS, messages=history)
    history.append({"role": "assistant",
                    "content": r.content})
    if r.stop_reason == "end_turn":
        break
    results = [run(b) for b in r.content
               if b.type == "tool_use"]
    history.append({"role": "user", "content": results})
```

Line 4

History keeps the tool_use turn

Line 6

`end_turn` is the only exit

Line 10

Results go back with matching ids

Claude keeps asking for the same tool

Script ran the test but never sent `tool_result`

Claude reads the result and moves on

`tool_use` turn, then `tool_result` with the same id

© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Watch one refund go round the loop

✗ Stop as soon as the model writes something that sounds finished.

→

✓ Continue on tool_use, stop on end_turn, append every result.

→

get_customer, lookup_order, recommend_refund, process_refund, then end_turn: refund $120.

Simulator 1
Customer Support Resolution Agent

Browser: Simulator 1, Lab 1 · Claude Code: /lab 1 · Terminal, in simulator-1-customer-support:

```
$ uv run support_agent.py --case CS-01
```


© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Claude re-asks because the tool_result never came back

✗ The script runs the test, then sends no matching tool_result.

→

✓ Append the tool_use turn, then a user turn with a matching tool_result.

→

You draw the three turns: prompt, tool_use with an id, tool_result with that same id.

Simulator 2
Code Generation with Claude Code

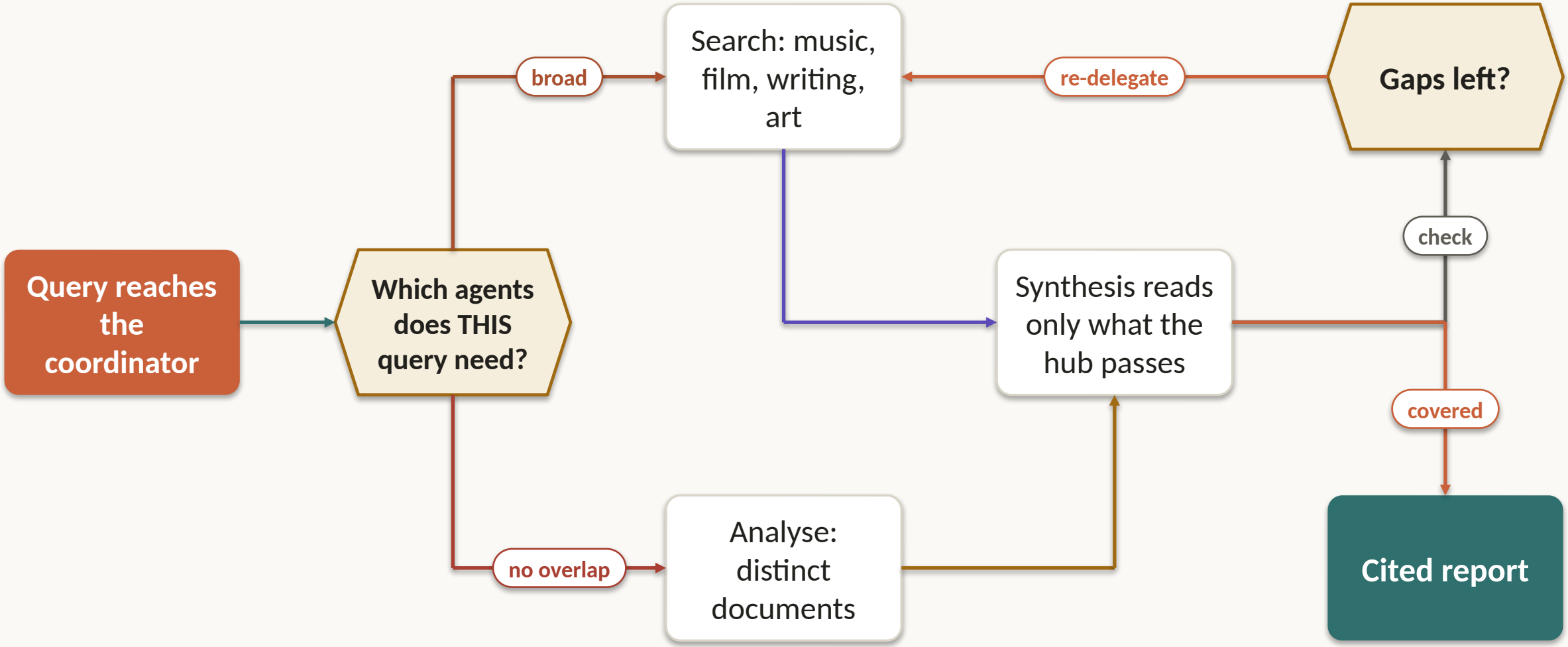
Browser: Simulator 2, Lab 2 · Claude Code: /lab 2 · Terminal, in simulator-2-code-generation:

```
$ uv run casekit.py --case UC-17
```


© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

One hub. Every message goes through it.

Subagents never talk to each other, so you can see, retry and reroute everything.



© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Every subagent succeeds, and the report still misses half the topic

✗ "Creative industries" decomposed into three flavours of visual art.

→

✓ Decompose broadly: music, writing, film and visual arts.

→

The narrow run never searches music, writing or film. The broad run does.

Simulator 3
Multi-Agent Research System

Browser: Simulator 3, Lab 3 · Claude Code: /lab 3 · Terminal, in simulator-3-multi-agent-research:

```
$ uv run research_system.py --narrow
$ uv run research_system.py
```


© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Same brief to every subagent, same findings four times

✗ All subagents hunt schema failures across all 600 documents.

→

✓ Give each one a distinct document group or concern, and demand concise findings.

→

Every document lands in exactly one group, and each subagent gets a line-capped deliverable.

Simulator 6
Structured Data Extraction

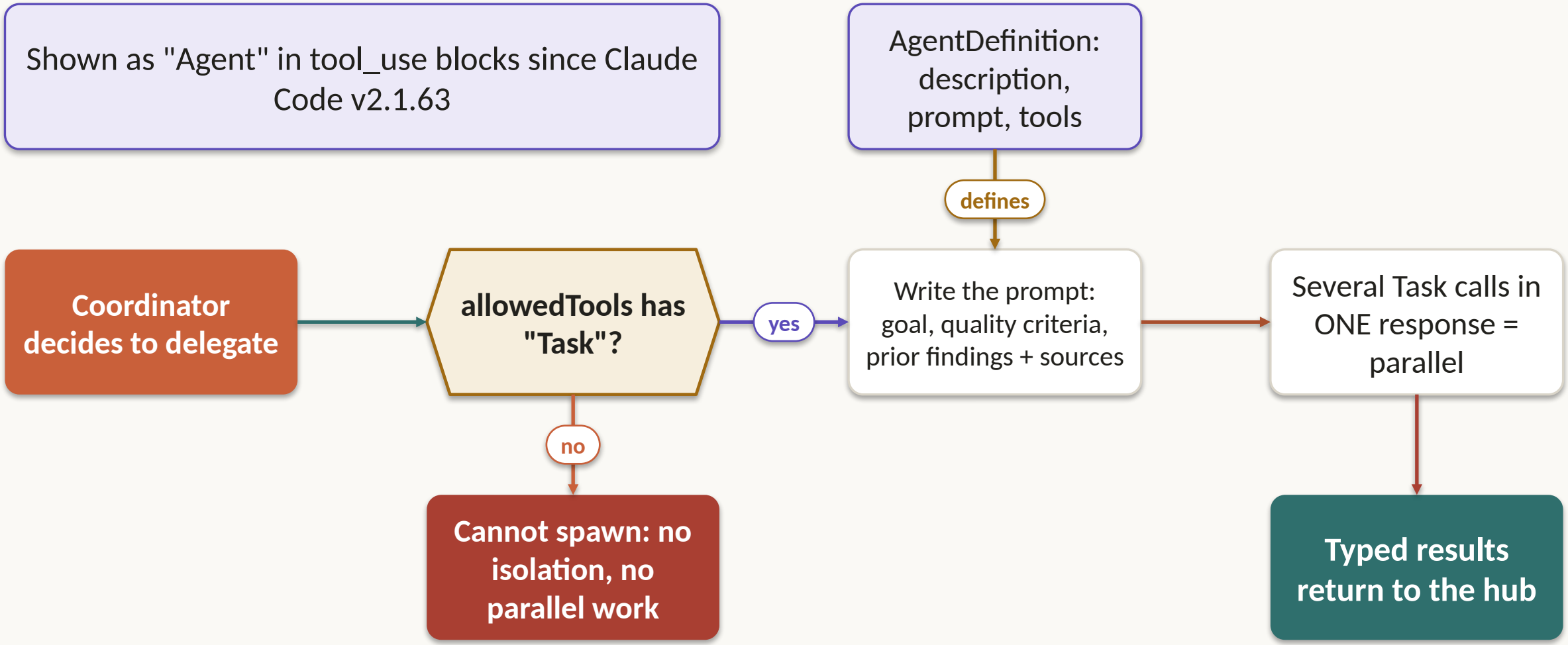
Browser: Simulator 6, Lab 4 · Claude Code: /lab 4 · Terminal, in simulator-6-structured-extraction:

```
$ uv run extract.py --case UC-36
```


© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

The Task tool is how a coordinator spawns anything

No "Task" in allowedTools means no subagents. A subagent sees only your prompt.



© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Subagents start with an empty head

Whatever they need, you paste into the prompt.

No Task in allowed_tools, no subagents

```
options = ClaudeAgentOptions(
    allowed_tools=["Task", "Read"],  # "Agent" since v2.1.63
    agents={"web-search": AgentDefinition(
        description="Searches one subtopic on the web",
        prompt="Return JSON claims with source and date.",
        tools=["search_web"])})
```

Spawn in parallel

Emit several Task calls in one response, not across turns

Brief with goals

State the research goal and quality bar, not a step list

What people assume

The subagent read the chat. It remembers the last run.

What actually happens

Empty context. Paste the complete findings and source metadata into its prompt.

© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

A /parallel-review that can't fan out because Task isn't allowed

✗ Only Read, Grep and Glob are auto-approved, so delegation stalls.

→

✓ Add Task to the coordinator's allowed tools.

→

Delegation disallowed: no reviewer spawns. Allowed: the subagent runs and its answer comes back.

Simulator 2
Code Generation with Claude Code

Browser: Simulator 2, Lab 5 · Claude Code: /lab 5 · Terminal, in simulator-2-code-generation:

```
$ uv run casekit.py --case UC-25
```


© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Same reviewers, same findings, and the PR comment loses them all

✗ Synthesis is told to "summarise the review" and inherits nothing.

→

✓ Paste each reviewer's complete JSON findings into the synthesis prompt.

→

File paths and failing test names survive with --pass-findings and vanish with --lossy.

Simulator 5
Claude Code for Continuous Integration

Browser: Simulator 5, Lab 6 · Claude Code: /lab 6 · Terminal, in simulator-5-ci-cd:

```
$ uv run ci_subagents.py --compare
```


© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Polished report, weak citations: route information, not just tasks

✗ Synthesis gets a one-line brief and no source details.

→

✓ Put findings and source metadata directly in the synthesis prompt.

→

The synthesis prompt runs to thousands of characters, with URLs, document names and dates.

Simulator 3
Multi-Agent Research System

Browser: Simulator 3, Lab 7 · Claude Code: /lab 7 · Terminal, in simulator-3-multi-agent-research:

```
$ uv run research_system.py --case UC-10
```


© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Later subagents re-run searches the first ones already finished

✗ Next-phase prompts carry no memory of rejected sources.

→

✓ Add a source-linked summary of prior phases to each delegation prompt.

→

Synthesis and report prompts carry the source links, far smaller than the raw subagent traffic.

Simulator 3
Multi-Agent Research System

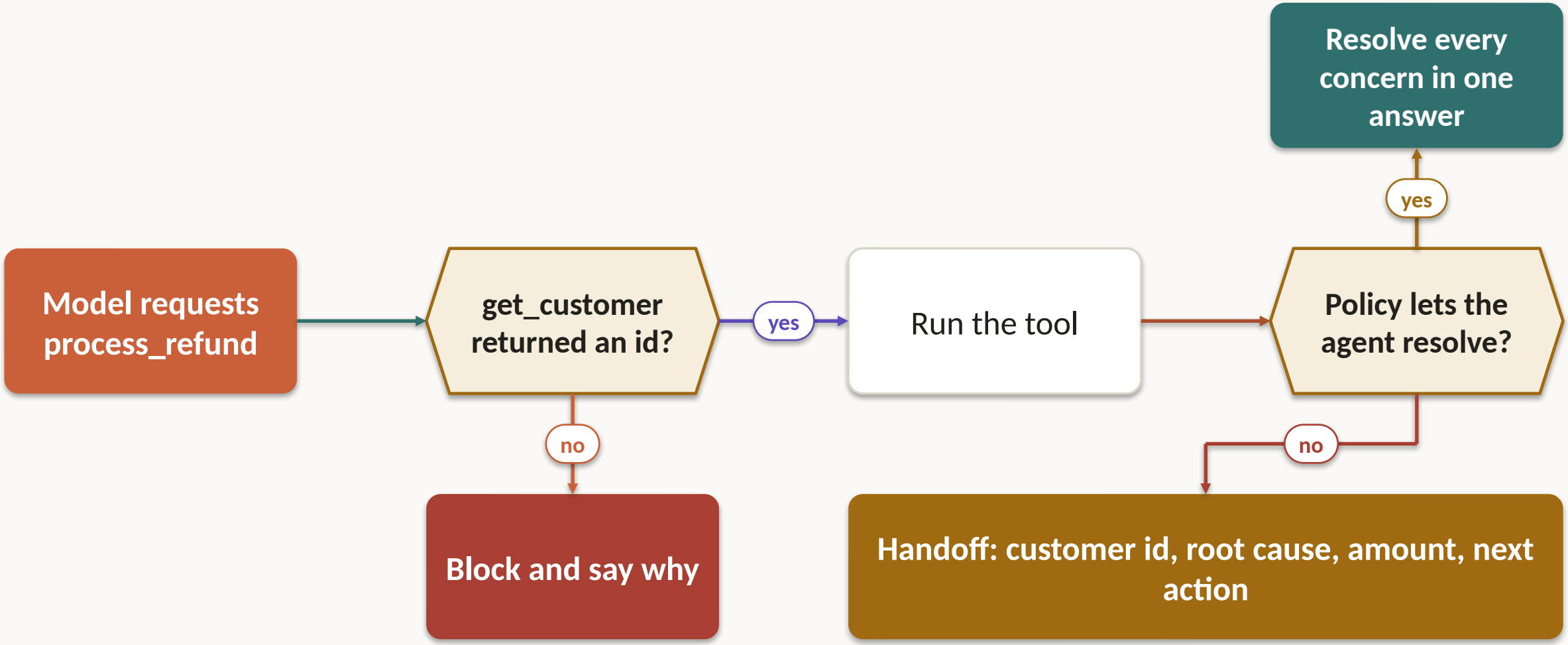
Browser: Simulator 3, Lab 8 · Claude Code: /lab 8 · Terminal, in simulator-3-multi-agent-research:

```
$ uv run research_system.py --case UC-35
```


© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Identity before refund is code, not a polite request

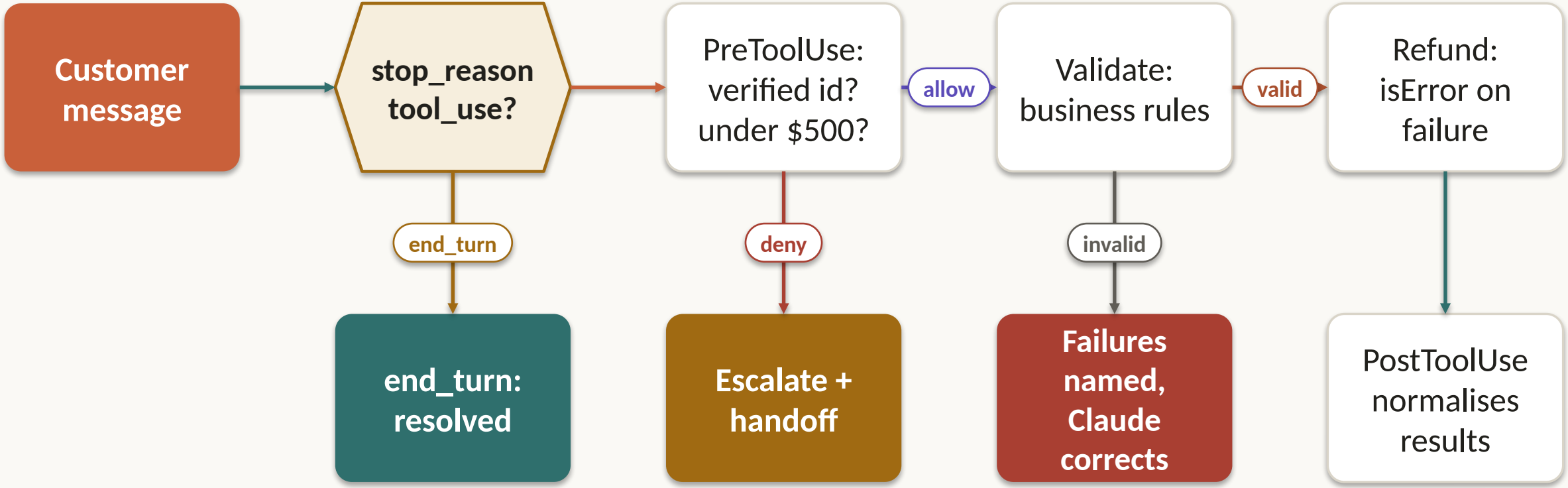
A prompt rule has a non-zero failure rate. A gate has none.



© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Follow one refund through the whole pipeline

stop_reason, PreToolUse, validation, errors and handoff, in one picture.



Task 1.1 Loop on stop_reason

Task 1.4 and 1.5 Hooks enforce order and limits

Task 2.2 and 4.4 Errors and validators feed back

© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

12% of cases skip get_customer: fix it with a gate, not a sentence

✗ A system prompt that says verification is mandatory.

→

✓ A prerequisite hook blocks lookup_order and process_refund until get_customer returns an id.

→

With the gate on, get_customer always comes first. Compare the two traces.

Simulator 1
Customer Support Resolution Agent

Browser: Simulator 1, Lab 9 · Claude Code: /lab 9 · Terminal, in simulator-1-customer-support:

```
$ uv run support_agent.py --case CS-03
$ uv run support_agent.py --case CS-03 --no-gate --tag nogate
```


© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Approval must never post before the passing test artifact exists

✗ A system prompt rule, obeyed in most pull requests.

→

✓ A PreToolUse command hook denies approval while the artifact is absent.

→

No artifact: exit 2, denied. Tests run first: PR 418 approved and logged.

Simulator 5
Claude Code for Continuous Integration

Browser: Simulator 5, Lab 10 · Claude Code: /lab 10 · Terminal, in simulator-5-ci-cd:

```
$ uv run hook_gate.py
```


© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Claude guesses a field and submits without validating

✗ Tool descriptions ask nicely for validate_record first.

→

✓ An application-level gate blocks submit_record until validation succeeds.

→

DOC-07 is blocked at submit_record, nothing submits unvalidated, and a review ticket opens.

Simulator 6
Structured Data Extraction

Browser: Simulator 6, Lab 11 · Claude Code: /lab 11 · Terminal, in simulator-6-structured-extraction:

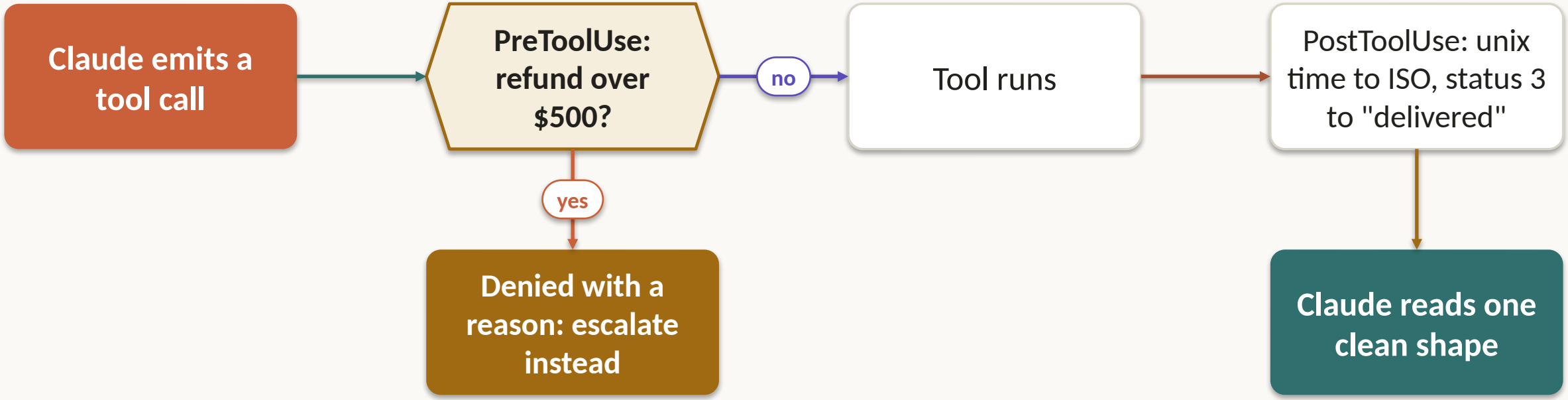
```
$ uv run extract.py --case UC-12
```


© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

PreToolUse decides. PostToolUse reshapes.

One hook stands before the tool, one after it, and neither depends on Claude's mood.

Business rule with money attached? Use a hook, never only a prompt.



© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

A hook is the only rule Claude can't talk its way past

Refund limits and identity checks belong here.

Eight lines of code beat a paragraph of policy

```
async def limit(data, tool_use_id, ctx):
    if data["tool_input"].get("amount", 0) > 500:
        return {"hookSpecificOutput": {
            "hookEventName": "PreToolUse",
            "permissionDecision": "deny",
            "permissionDecisionReason":
                "Over $500: escalate to a human."}}
    return {}
```

$780

refund blocked even when the prompt omits the rule

0

unverified refunds with the gate on

Prompt: "always verify first"

Usually obeyed. Some calls skip it.

→

Hook: permissionDecision deny

Same outcome on every call, with the reason fed back to Claude

© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

The prompt forgot the $500 rule. The hook did not.

✗ A refund limit that lives only in the system prompt.

→

✓ A PreToolUse hook denies anything above $500 and redirects to escalation.

→

Without the rule in the prompt the model tries $780. The hook denies it.

Simulator 1
Customer Support Resolution Agent

Browser: Simulator 1, Lab 12 · Claude Code: /lab 12 · Terminal, in simulator-1-customer-support:

```
$ uv run support_agent.py --case CS-02
$ uv run support_agent.py --case CS-02 --prompt prompts/system-noguard.md --tag noguard
```


© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Shrink noisy npm test output before Claude reads it

✗ Rewrite the command, or tell Claude to ignore the noise.

→

✓ A PostToolUse hook on Bash returns updated output, leaving the command as is.

→

The command still runs. Claude reads no npm WARN lines and no absolute paths.

Simulator 2
Code Generation with Claude Code

Browser: Simulator 2, Lab 13 · Claude Code: /lab 13 · Terminal, in simulator-2-code-generation:

```
$ uv run casekit.py --case UC-9
```


© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

rm -rf meets a hook, not a polite request

✗ A CLAUDE.md line saying "be careful with deletes".

→

✓ A PreToolUse hook denies Bash commands containing rm -rf.

→

The hook denies the command, work/tests still exists, and the agent reports the policy.

Simulator 4
Developer Productivity with Claude

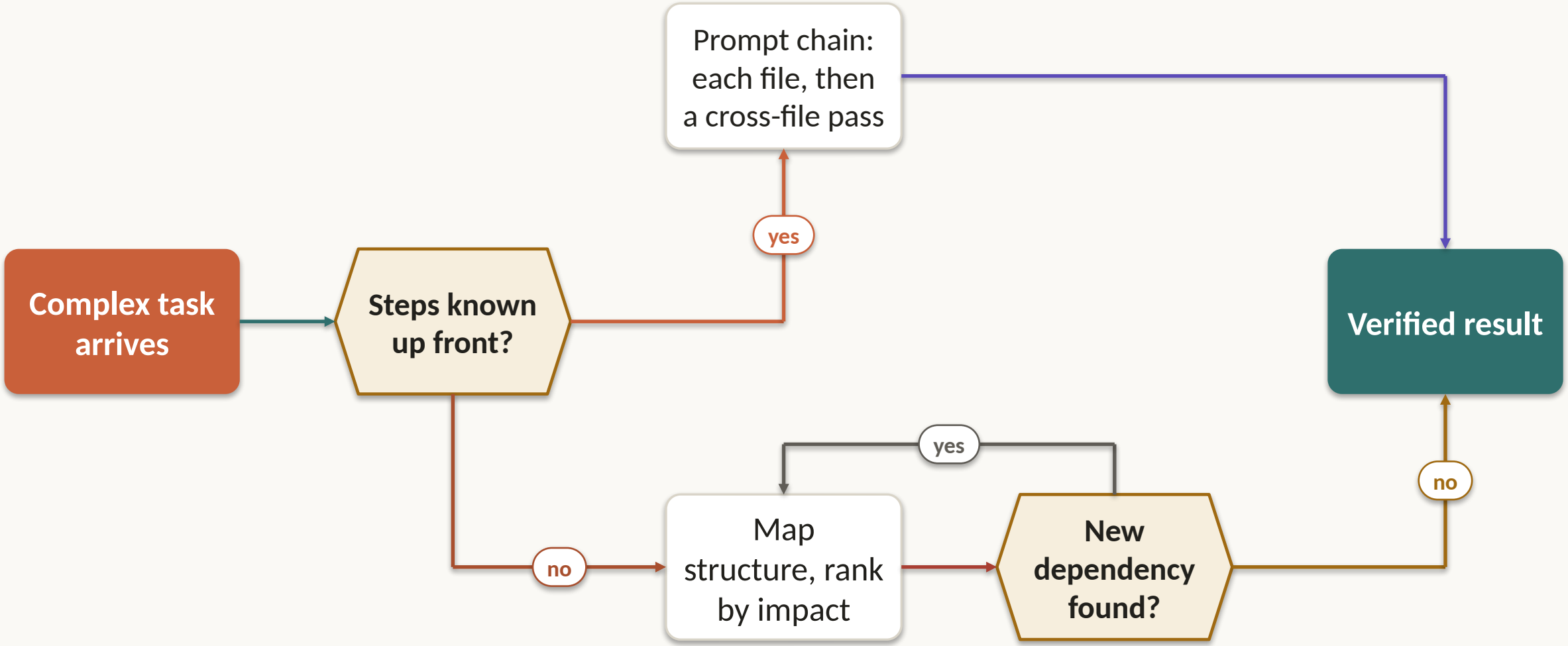
Browser: Simulator 4, Lab 14 · Claude Code: /lab 14 · Terminal, in simulator-4-developer-productivity:

```
$ uv run explorer.py --case UC-D6
```


© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Known steps get a chain. Unknown ground gets a plan that adapts.

Pick by whether you can list the steps before you start.



© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

60-page packets: section passes first, reconcile second

✗ One giant single pass over every section.

→

✓ Extract per section, then a reconciliation pass checks totals and duplicates.

→

Single pass: one record for three documents. Section pipeline: one record per section.

Simulator 6
Structured Data Extraction

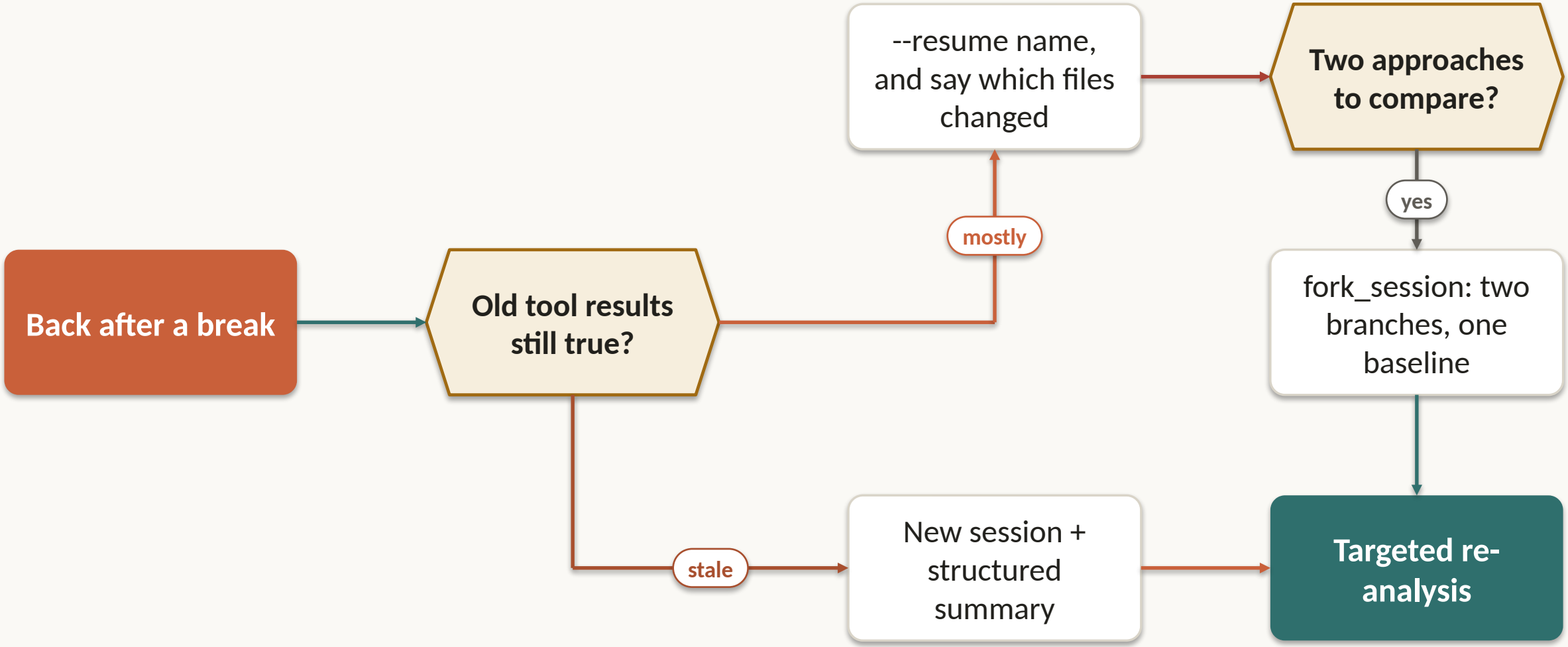
Browser: Simulator 6, Lab 15 · Claude Code: /lab 15 · Terminal, in simulator-6-structured-extraction:

```
$ uv run extract.py --case UC-31
```


© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Resume when the facts still hold. Start fresh when they don't.

A resumed session trusts its old tool results, even when the code has moved on.



© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Two attempts, one analysis: fork the session, checkpoint the files

✗ Resume the same session twice in one working folder.

→

✓ Resume with the fork option for each attempt, and checkpoint files first.

→

Three session ids. Attempt B starts from restored files, so attempt A never leaks in.

Simulator 2
Code Generation with Claude Code

Browser: Simulator 2, Lab 16 · Claude Code: /lab 16 · Terminal, in simulator-2-code-generation:

```
$ uv run casekit.py --case UC-5
```


© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Resume, and say exactly which file changed

✗ Resume silently and let the agent trust stale tool results.

→

✓ Name the changed file, or start fresh with a structured summary.

→

Same session id, at most two Reads, no Grep, no subagent.

Simulator 4
Developer Productivity with Claude

Browser: Simulator 4, Lab 17 · Claude Code: /lab 17 · Terminal, in simulator-4-developer-productivity:

```
$ uv run explorer.py --case UC-D4
```


© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

TOPIC 2 · EXAM DOMAIN 2 · 18% OF THE EXAM

Tool Design & MCP Integration

Tools are the agent's only hands. Describe them well, fail them clearly.



© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

TOPIC 2 MAP

5 task statements, 9 labs, 6 simulators

Concept first, then its labs, straight away.

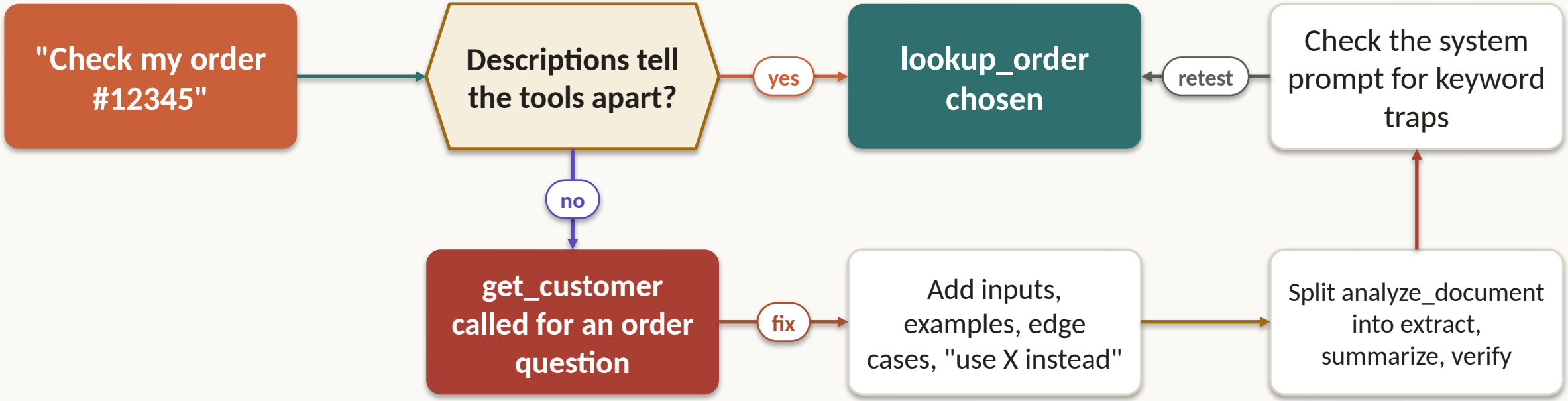
Task statement	Sim 1	Sim 2	Sim 3	Sim 4	Sim 5	Sim 6
2.1 Design effective tool interfaces with clear descriptions and boundaries						18
2.2 Implement structured error responses for MCP tools	19,20					
2.3 Distribute tools appropriately across agents and configure tool choice			22		21	
2.4 Integrate MCP servers into Claude Code and agent workflows		23		24		
2.5 Select and apply built-in tools (Read, Write, Edit, Bash, Grep, Glob) effectively				25,26		

© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

The description is the router

Claude picks tools from what you wrote about them. Say little and it guesses.

analyze_content vs analyze_document: near-identical text, constant misrouting



© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

One analyze_document tool does three jobs badly

✗ A single tool with an operation parameter.

→

✓ A few non-overlapping tools with clear descriptions and JSON Schema inputs.

→

The generic tool drifts into summaries. The split set calls extract_invoice with valid fields.

Simulator 6
Structured Data Extraction

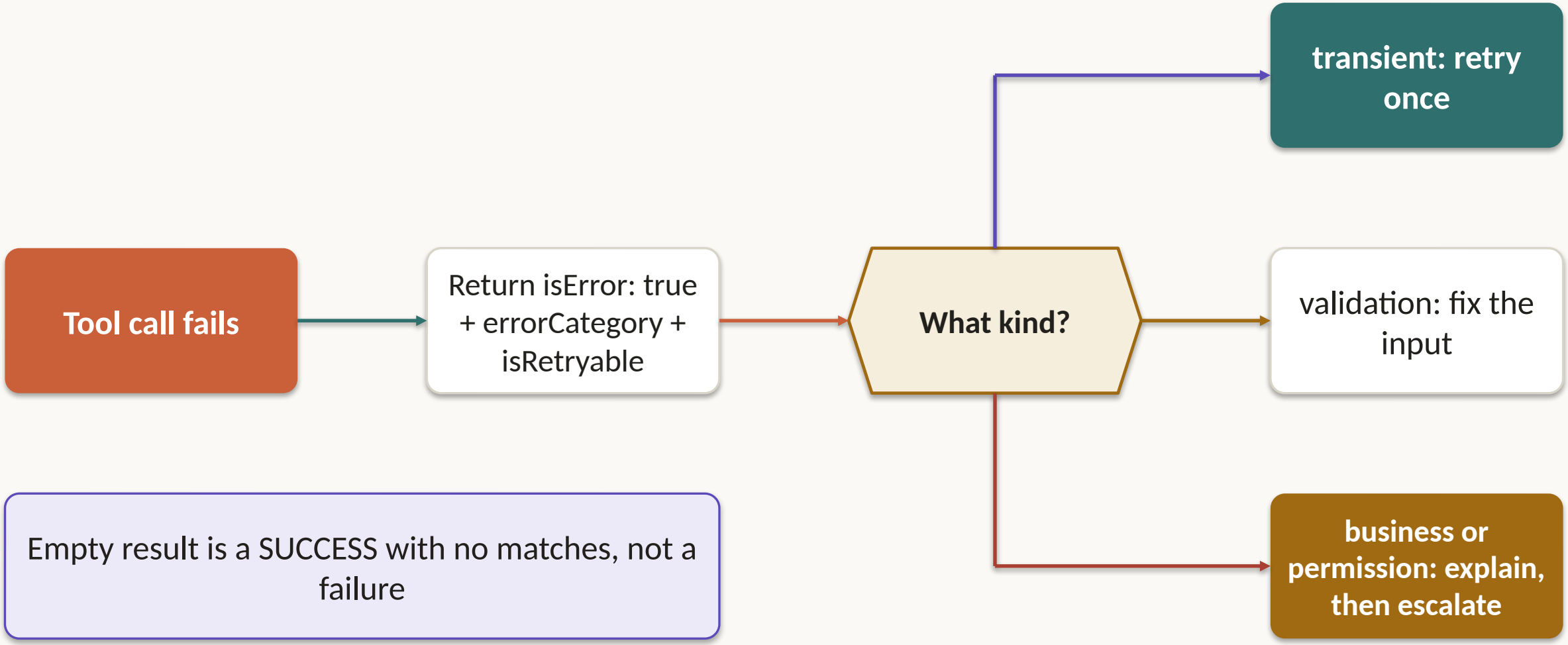
Browser: Simulator 6, Lab 18 · Claude Code: /lab 18 · Terminal, in simulator-6-structured-extraction:

```
$ uv run extract.py --case UC-11
```


© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

"Operation failed" tells the agent nothing it can use

Name the failure, and the agent knows whether to retry, explain or escalate.



© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Five outcomes, five different next moves

A flat "failed" collapses all of them into a blind retry.

errorCategory	Example	isRetryable	What the agent does
transient	Timeout, service down	true	Retry once, then report what it tried
validation	Malformed order id	false	Fix the input
business	Outside the return window	false	Explain kindly, offer the next option
permission	Account not authorised	false	Escalate
no error	Query ran, 0 matches	n/a	Say nothing matched. Never retry.

© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

The refund failed. The customer was told it worked.

✗ Failure text returned as a normal, successful tool result.

→

✓ Return isError: true with structured failure details.

→

The refund on ORD-5508 is never issued, and the reply never claims it was.

Simulator 1
Customer Support Resolution Agent

Browser: Simulator 1, Lab 19 · Claude Code: /lab 19 · Terminal, in simulator-1-customer-support:

```
$ uv run support_agent.py --case CS-11
```


© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Retry the timeout once, then answer from the real record

✗ A generic "lookup failed" with no hint about retrying.

→

✓ errorCategory plus isRetryable lets the agent decide.

→

ORD-5504 times out once, one retry follows, and the reply uses the real order record.

Simulator 1
Customer Support Resolution Agent

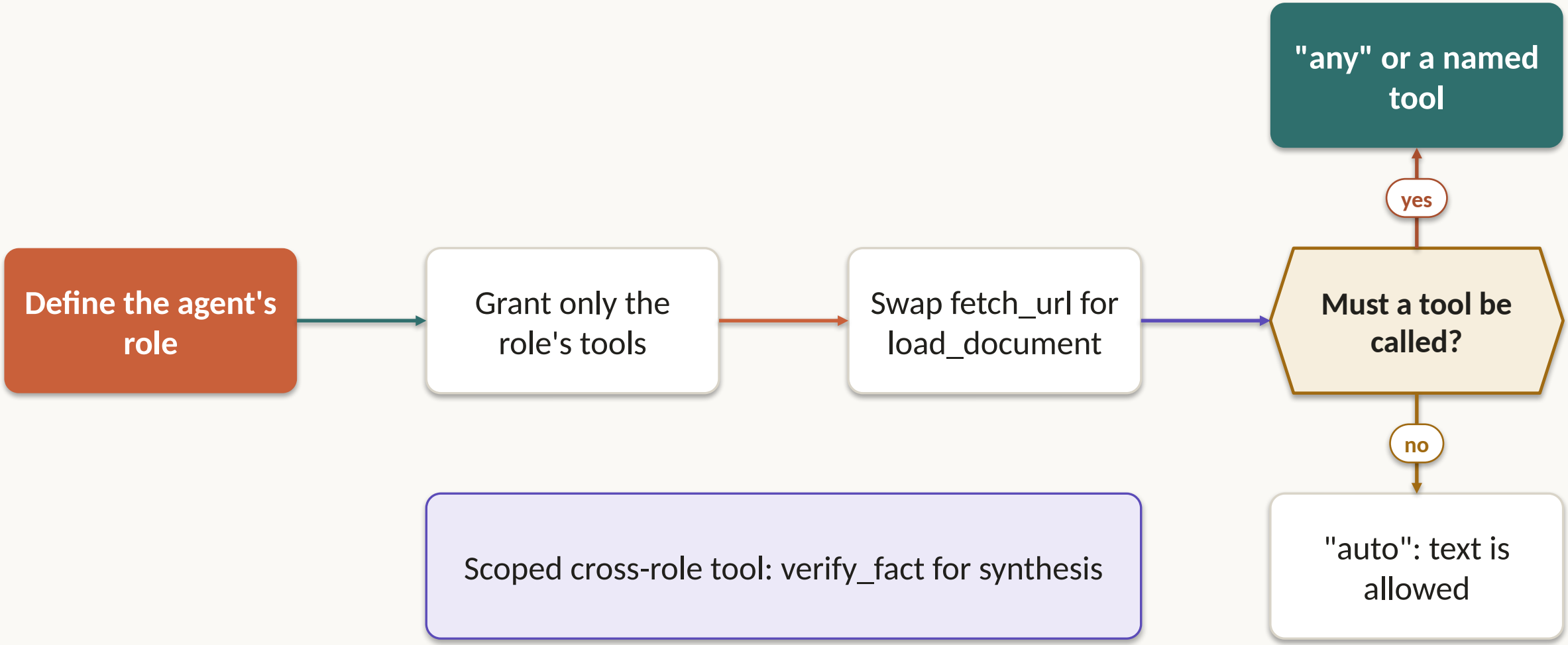
Browser: Simulator 1, Lab 20 · Claude Code: /lab 20 · Terminal, in simulator-1-customer-support:

```
$ uv run support_agent.py --case CS-06
```


© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

18 tools degrade selection. Give each agent 4 or 5.

A synthesis agent holding a web-search tool will use it. Don't hand it one.



© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

tool_choice decides who has the last word

Force the first step, then let Claude pick the rest in later turns.

tool_choice	What it forces	Use it when
"auto"	May answer in plain text. No guarantee.	Normal agent turns
"any"	Must call a tool, Claude picks which	Several schemas, document type unknown
{"type": "tool", "name": ...}	Must call this exact tool	extract_metadata has to run first

18 to 5

tools per agent: selection gets reliable

Current API note

Opus 5.5, Sonnet 5.5 and Fable 5.1 answer `any` and forced with a 400. Use `auto` + `strict` + a prompt line. The exam still tests all three options.

© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Review and test generation fight over one Claude call

✗ All tools exposed to one step that also writes tests.

→

✓ Separate steps, role-specific tools, a forced reporting tool for review.

→

Review gets read-only tools and the finding schema. Tests get their own. No prose comes back.

Simulator 5
Claude Code for Continuous Integration

Browser: Simulator 5, Lab 21 · Claude Code: /lab 21 · Terminal, in simulator-5-ci-cd:

```
$ uv run casekit.py --case UC-15
```


© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Give synthesis one narrow tool, not the whole web

✗ Synthesis routes every small fact check through the coordinator.

→

✓ A scoped verify_fact tool for simple lookups. Complex cases still go through the hub.

→

Synthesis calls verify_fact. No agent uses a tool outside its definition.

Simulator 3
Multi-Agent Research System

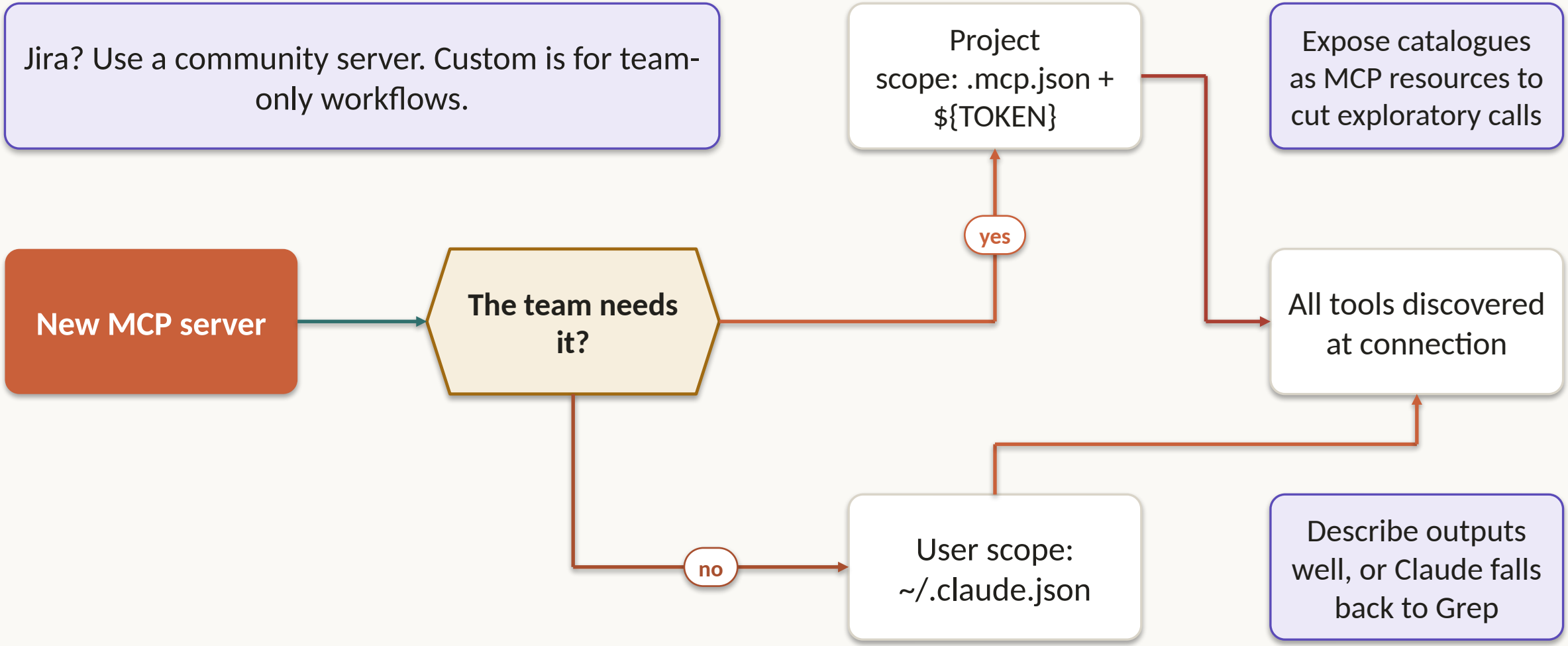
Browser: Simulator 3, Lab 22 · Claude Code: /lab 22 · Terminal, in simulator-3-multi-agent-research:

```
$ uv run research_system.py
```


© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Shared servers live in the repo. Secrets never do.

Commit .mcp.json with ${GITHUB_TOKEN}. Teammates get the tool, nobody gets your key.



© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Commit the server, never the token

✗ A shared .mcp.json with a literal secret, or a server only you have.

→

✓ ${GITHUB_TOKEN} in the project file, experiments in user scope.

→

.mcp.json holds placeholders only. Your real ~/.claude.json stays untouched.

Simulator 2
Code Generation with Claude Code

Browser: Simulator 2, Lab 23 · Claude Code: /lab 23 · Terminal, in simulator-2-code-generation:

```
$ uv run simulator.py 5
```


© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Read the catalogue before opening a single file

✗ The agent greps blindly for the billing runbook.

→

✓ Expose a resource catalogue so it knows what exists first.

→

list_catalog first, read_resource docs://runbook next, no Glob over docs/.

Simulator 4
Developer Productivity with Claude

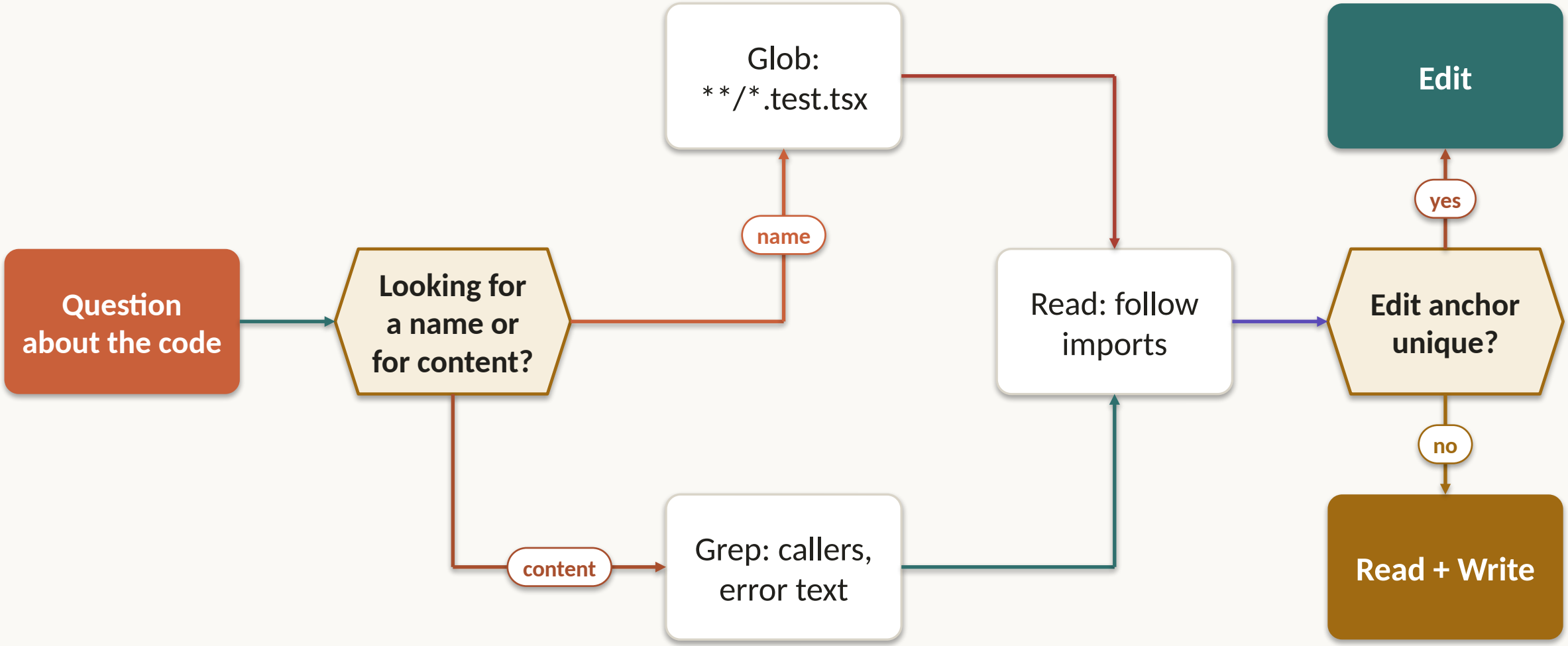
Browser: Simulator 4, Lab 24 · Claude Code: /lab 24 · Terminal, in simulator-4-developer-productivity:

```
$ uv run explorer.py --case UC-D2
```


© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Search narrows. Read confirms. Edit changes.

Reading every file up front is how you burn the context window on day one.



© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Six needs, six built-in tools

Grep through Bash works. It also loses you the exam mark.

You need	Reach for	Example
A file by name	Glob	`**/*.test.tsx`
Text inside files	Grep	All callers of `calculate_total`, in `**/*.ts`
The whole file	Read	Follow the imports from the Grep hit
A small change	Edit	Needs a unique anchor
Edit said "not unique"	Read + Write	Load the file, write it back whole
Run something	Bash	`pytest -q`, never `grep` through Bash

© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

/find-callers needs Grep with a glob, not Bash

✗ Glob for content, or grep through Bash.

→

✓ Grep with the helper name, a file-type glob, output_mode content.

→

One Grep call for calculate_total in *.py, with file, line and match. No Bash grep.

Simulator 4
Developer Productivity with Claude

Browser: Simulator 4, Lab 25 · Claude Code: /lab 25 · Terminal, in simulator-4-developer-productivity:

```
$ uv run explorer.py --case UC-37
```


© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Edit says the anchor isn't unique. Now what?

✗ Retry Edit with the same anchor.

→

✓ Read the file, then Write it back whole.

→

return total appears twice. Edit is tried, the fallback or a wider anchor lands, pytest passes.

Simulator 4
Developer Productivity with Claude

Browser: Simulator 4, Lab 26 · Claude Code: /lab 26 · Terminal, in simulator-4-developer-productivity:

```
$ uv run explorer.py --case UC-D1
```


© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

TOPIC 3 · EXAM DOMAIN 3 · 20% OF THE EXAM

Claude Code Configuration & Workflows

Configure once, and every teammate and CI job gets the same Claude.



© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

TOPIC 3 MAP

6 task statements, 11 labs, 4 simulators

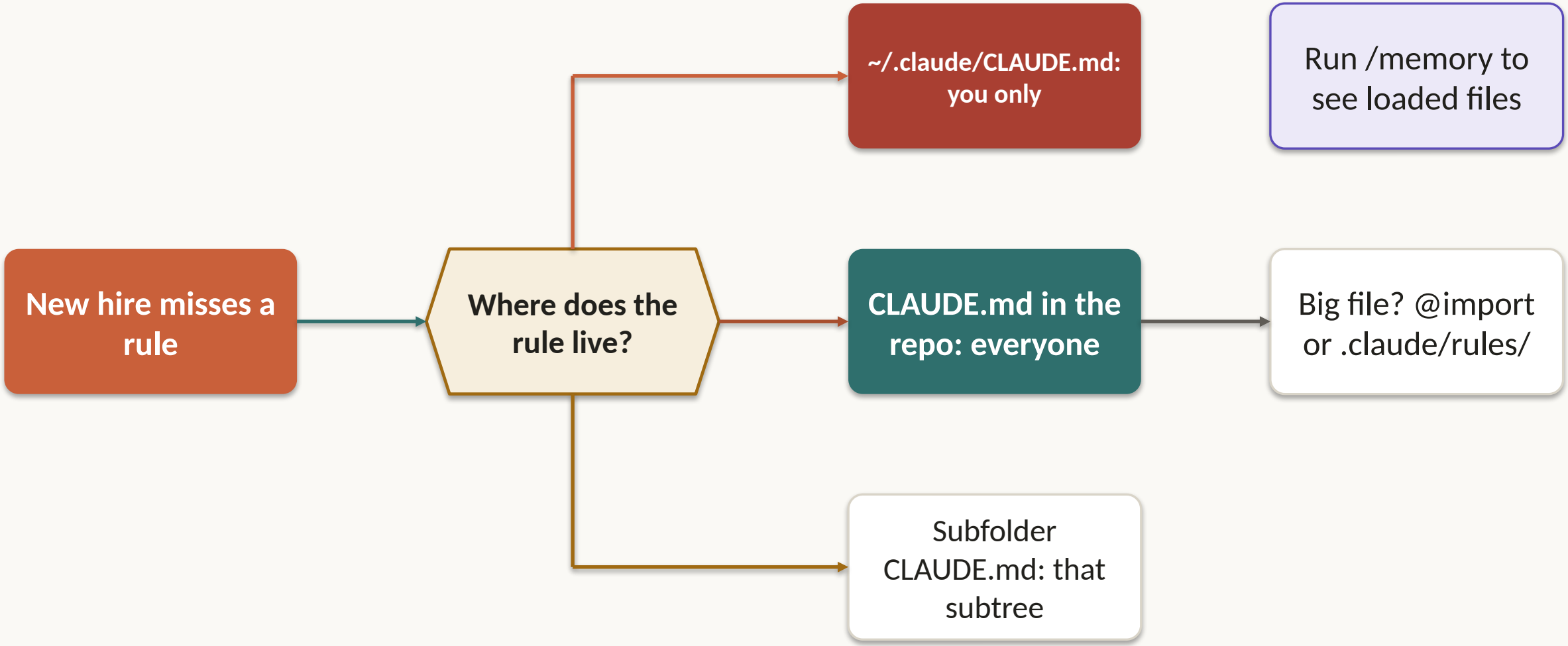
Concept first, then its labs, straight away.

Task statement	Sim 1	Sim 2	Sim 3	Sim 4	Sim 5	Sim 6
3.1 Configure CLAUDE.md files with appropriate hierarchy, scoping, and modular organization		27				
3.2 Create and configure custom slash commands and skills	29	28				
3.3 Apply path-specific rules for conditional convention loading			30			
3.4 Determine when to use plan mode vs direct execution		31,32,33				
3.5 Apply iterative refinement techniques for progressive improvement			35		34	
3.6 Integrate Claude Code into CI/CD pipelines					36,37	

© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

If it's in ~/.claude, your teammate never sees it

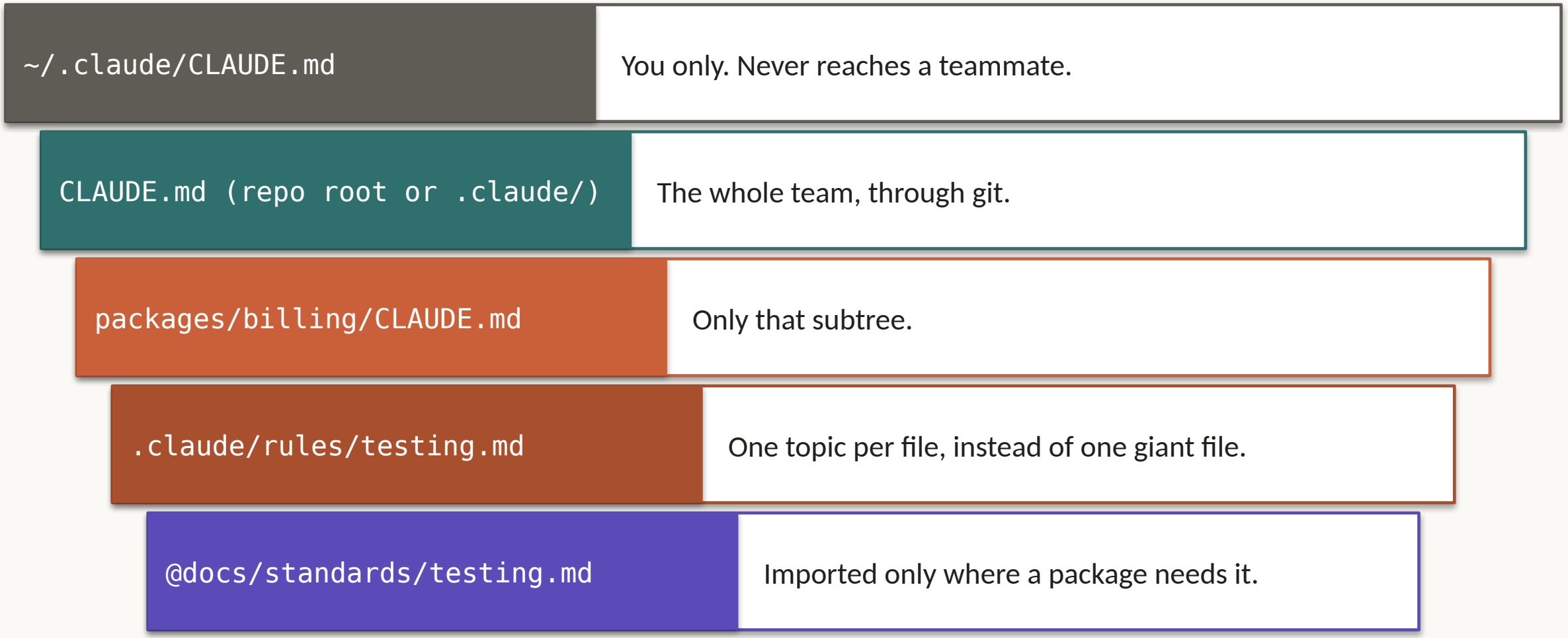
"Works on my machine" now applies to instructions. /memory shows what loaded.



© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Five places an instruction can live

Each layer answers one question: who should see this?



© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Which instructions actually reached Claude?

✗ Everything in one huge file, or hidden in your home folder.

→

✓ Project CLAUDE.md with @imports, plus rules that load by path.

→

Claude quotes the imported standard and names the rule files in context.

Simulator 2
Code Generation with Claude Code

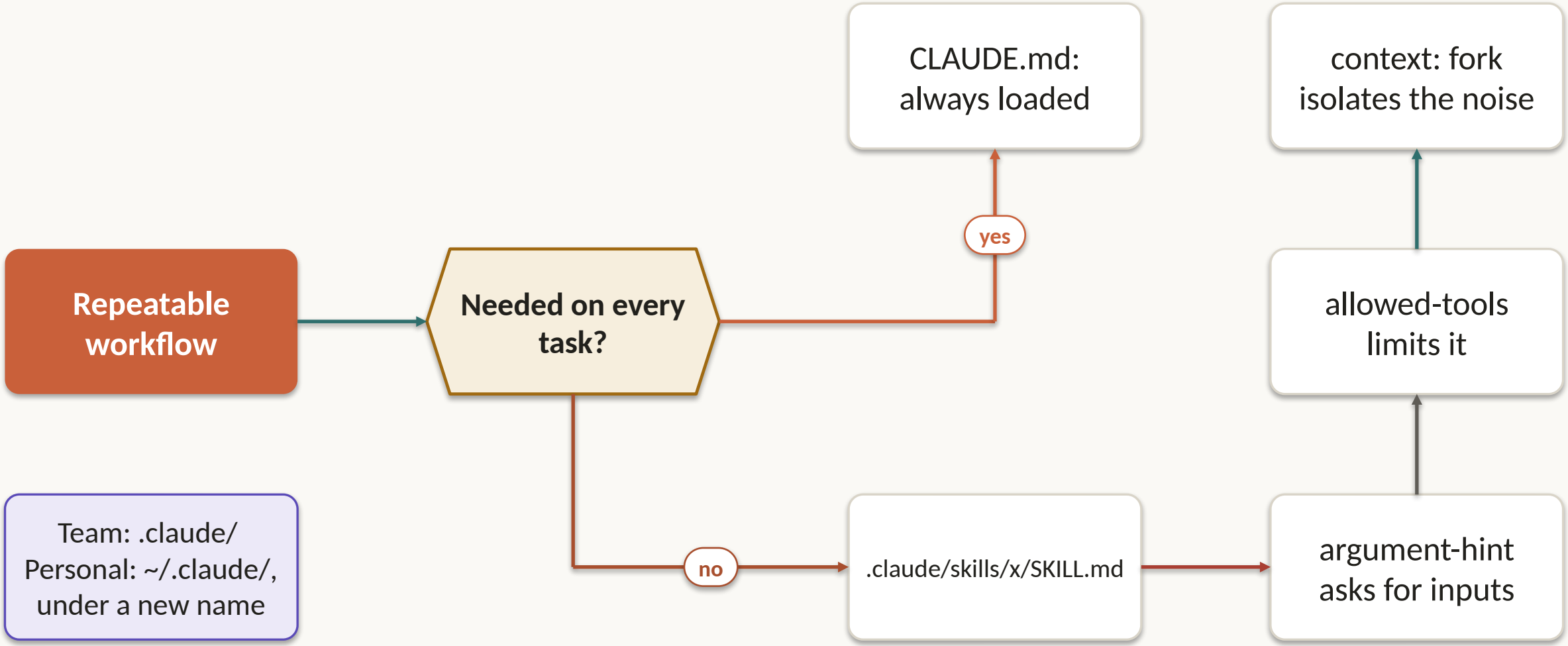
Browser: Simulator 2, Lab 27 · Claude Code: /lab 27 · Terminal, in simulator-2-code-generation:

```
$ uv run simulator.py 1
```


© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Skills run on demand. CLAUDE.md is always on.

Put verbose workflows in a skill with context: fork, and your main session stays clean.



© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

A /review command the whole team gets through git

✗ A personal command in your home folder.

→

✓ Commit it to .claude/commands/.

→

/review main finds the removed orders:read check and ends with a VERDICT line.

Simulator 2
Code Generation with Claude Code

Browser: Simulator 2, Lab 28 · Claude Code: /lab 28 · Terminal, in simulator-2-code-generation:

```
$ uv run simulator.py 2
```


© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Keep a noisy case review out of your main session

✗ A command or CLAUDE.md note that fills the conversation.

→

✓ SKILL.md with context: fork, allowed-tools and an argument-hint.

→

The review returns as a summary. The evidence JSON stays out of the session.

Simulator 1
Customer Support Resolution Agent

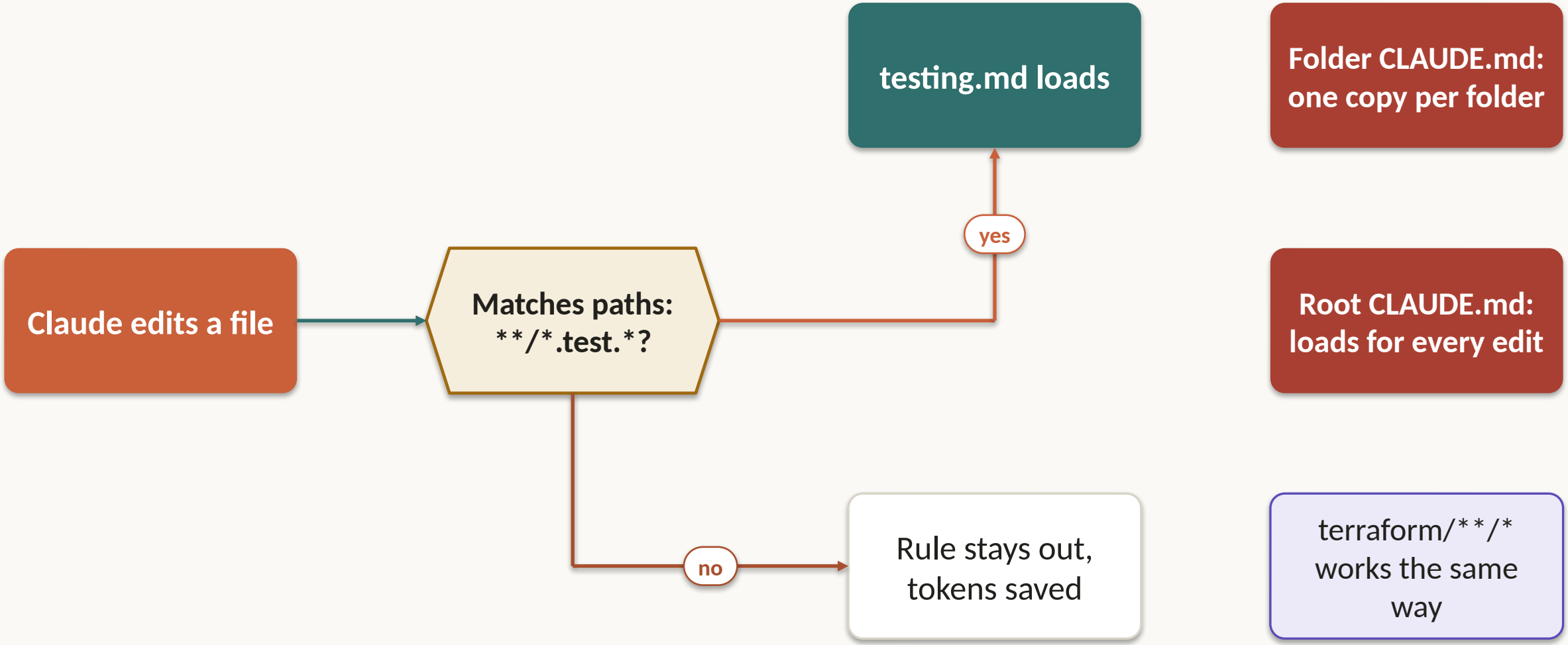
Browser: Simulator 1, Lab 29 · Claude Code: /lab 29 · Terminal, in simulator-1-customer-support:

```
$ uv run case_review.py CS-02
```


© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Test files live everywhere, so scope the rule by glob

A paths: pattern follows the file. A folder CLAUDE.md stays stuck in its folder.



© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Four designs, one winner, measured

We ran all four through Claude Code and watched what loaded.

.claude/rules/testing.md

```
---
paths: ["**/*.test.*"]
---
Use the shared fixtures.
One assertion per test.
```

Where the guidance lives	Test edit	Other edit	Copies
A Rule, no frontmatter	loads	loads	1
B Root CLAUDE.md	loads	loads	1
C CLAUDE.md per folder	loads	loads	3
D Rule with paths frontmatter	loads	stays out	1

What the simulator measured

Eight `claude -p` runs. Only variant D keeps the testing guidance out of the implementation edit while holding one copy. That is answer D on practice question 7.

© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Testing rules for test files spread across every folder

✗ Root CLAUDE.md, or one copy per folder.

→

✓ One rule file with paths frontmatter matching the test patterns.

→

D loads for the test edit only, one copy. A and B load always. C needs three copies.

Simulator 3
Multi-Agent Research System

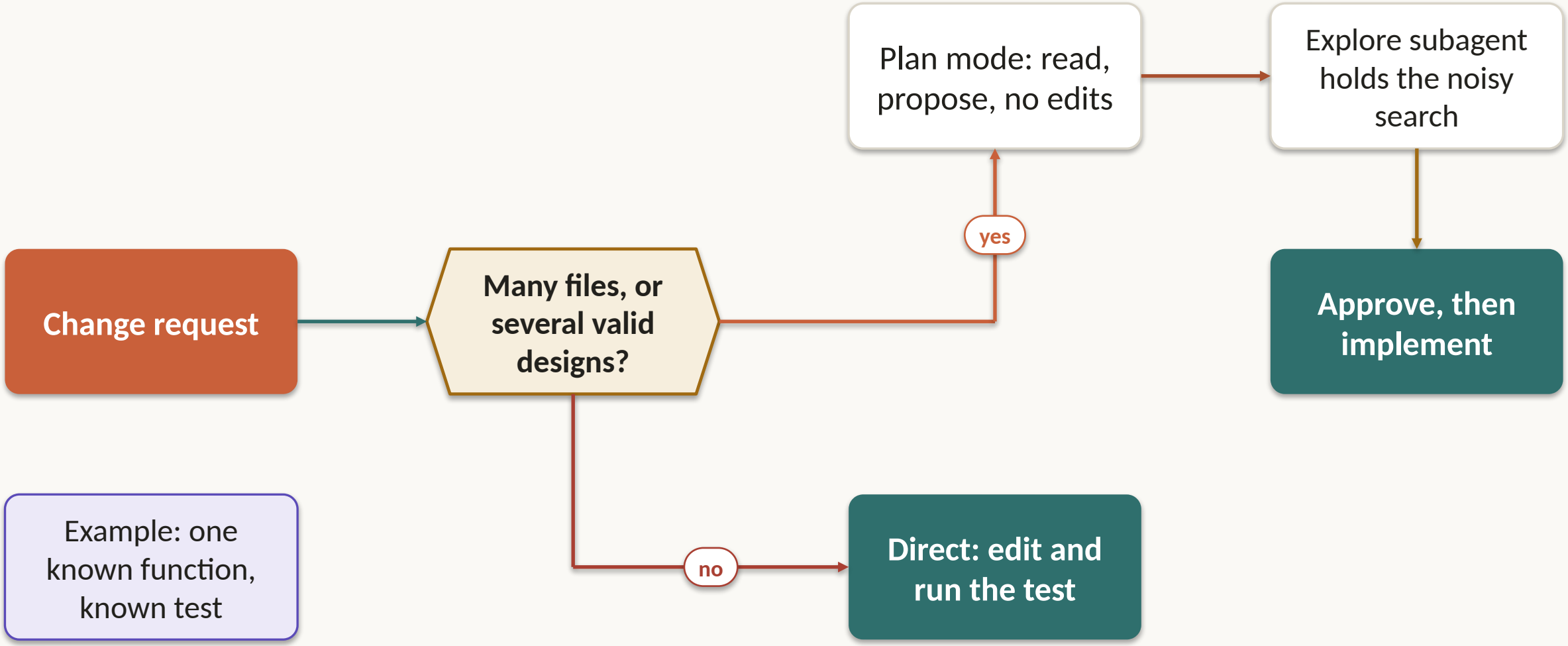
Browser: Simulator 3, Lab 30 · Claude Code: /lab 30 · Terminal, in simulator-3-multi-agent-research:

```
$ uv run rules_sim.py D
$ uv run rules_sim.py
```


© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Plan mode for blast radius. Direct for a known fix.

A guard clause in one function needs no plan. An auth migration across 45 files does.



© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Read the request, then pick the mode

The exam loves this distinction. So will your reviewers.

Plan mode first

Library migration across 45+ files

Microservice restructure

Engineers disagree on the boundary

Previous edits got reverted

or

Direct execution

One guard clause in a known function

Single-file fix with a clear stack trace

A date validation conditional

Existing test proves it

© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Replacing the auth layer: nobody agrees on the boundary, so plan

✗ Start editing and see what breaks.

→

✓ Plan mode researches and proposes before any edit.

→

Files hash the same before and after: a plan, with no edits.

Simulator 2
Code Generation with Claude Code

Browser: Simulator 2, Lab 31 · Claude Code: /lab 31 · Terminal, in simulator-2-code-generation:

```
$ uv run casekit.py --case UC-8
```


© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Plan first, then approve into an editing mode

✗ Immediate edits, which earlier caused broad rewrites.

→

✓ Plan mode, review the plan, approve, then edit.

→

Plan phase changes nothing. The same session resumes with acceptEdits, and the edits land.

Simulator 2
Code Generation with Claude Code

Browser: Simulator 2, Lab 32 · Claude Code: /lab 32 · Terminal, in simulator-2-code-generation:

```
$ uv run casekit.py --case UC-30
```


© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

One guard clause doesn't need a plan

✗ Plan mode and a design review for one line.

→

✓ Ask directly, then run the named test.

→

No plan mode. quantity.js changes, node --test runs, three tests pass.

Simulator 2
Code Generation with Claude Code

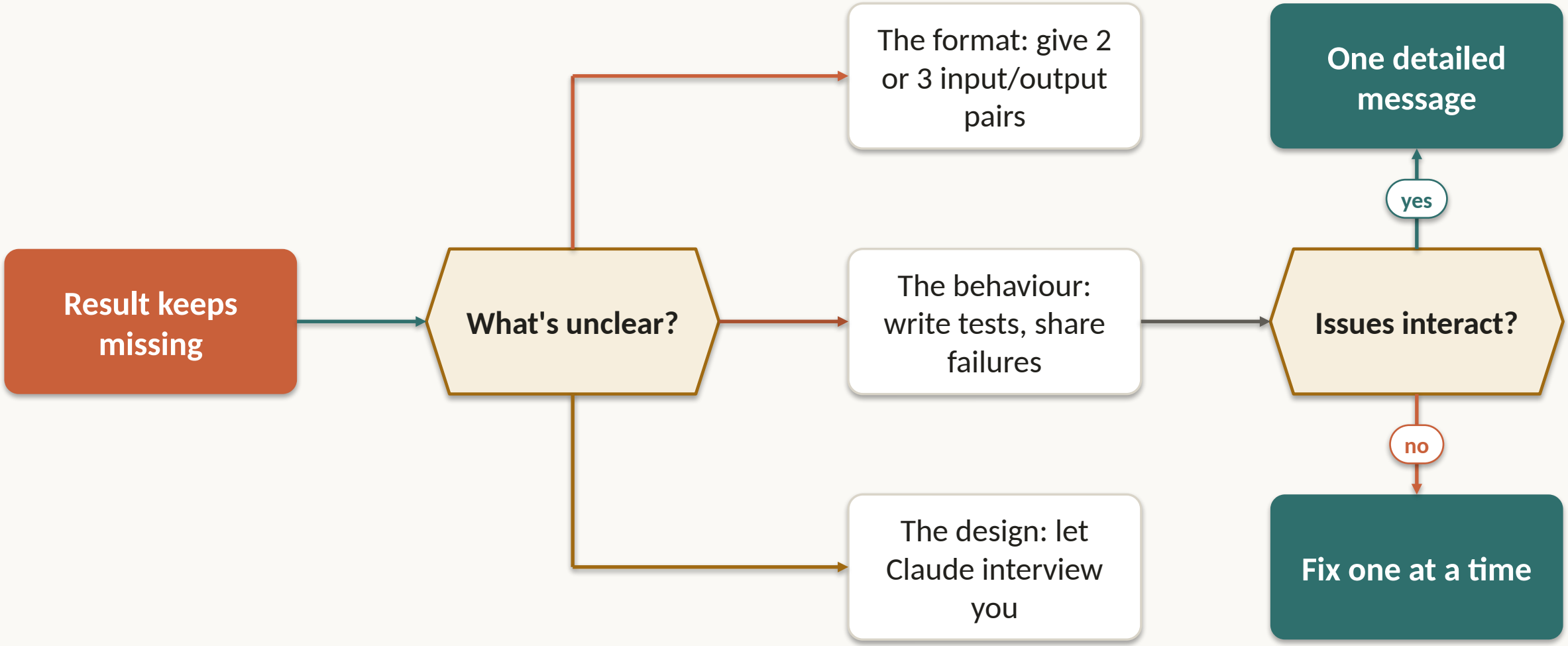
Browser: Simulator 2, Lab 33 · Claude Code: /lab 33 · Terminal, in simulator-2-code-generation:

```
$ uv run casekit.py --case UC-34
```


© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Show the output you want. Prose gets interpreted.

Examples, failing tests and questions from Claude beat a longer paragraph when prose keeps getting misread.



© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Unknown requirements? Let Claude interview you first

✗ Ask for the implementation and iterate blind.

→

✓ Interview, write a self-contained SPEC.md, implement in a fresh session.

→

spec.md holds open questions, interfaces, out-of-scope items and verification.

Simulator 5
Claude Code for Continuous Integration

Browser: Simulator 5, Lab 34 · Claude Code: /lab 34 · Terminal, in simulator-5-ci-cd:

```
$ uv run casekit.py --case UC-14
```


© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Two linked defects go in one detailed request

✗ Fix citations now, excerpts later, in separate runs.

→

✓ Plan mode with one request naming both defects and the checks.

→

You write one plan-mode request: both defects, shared interface, acceptance test.

Simulator 3
Multi-Agent Research System

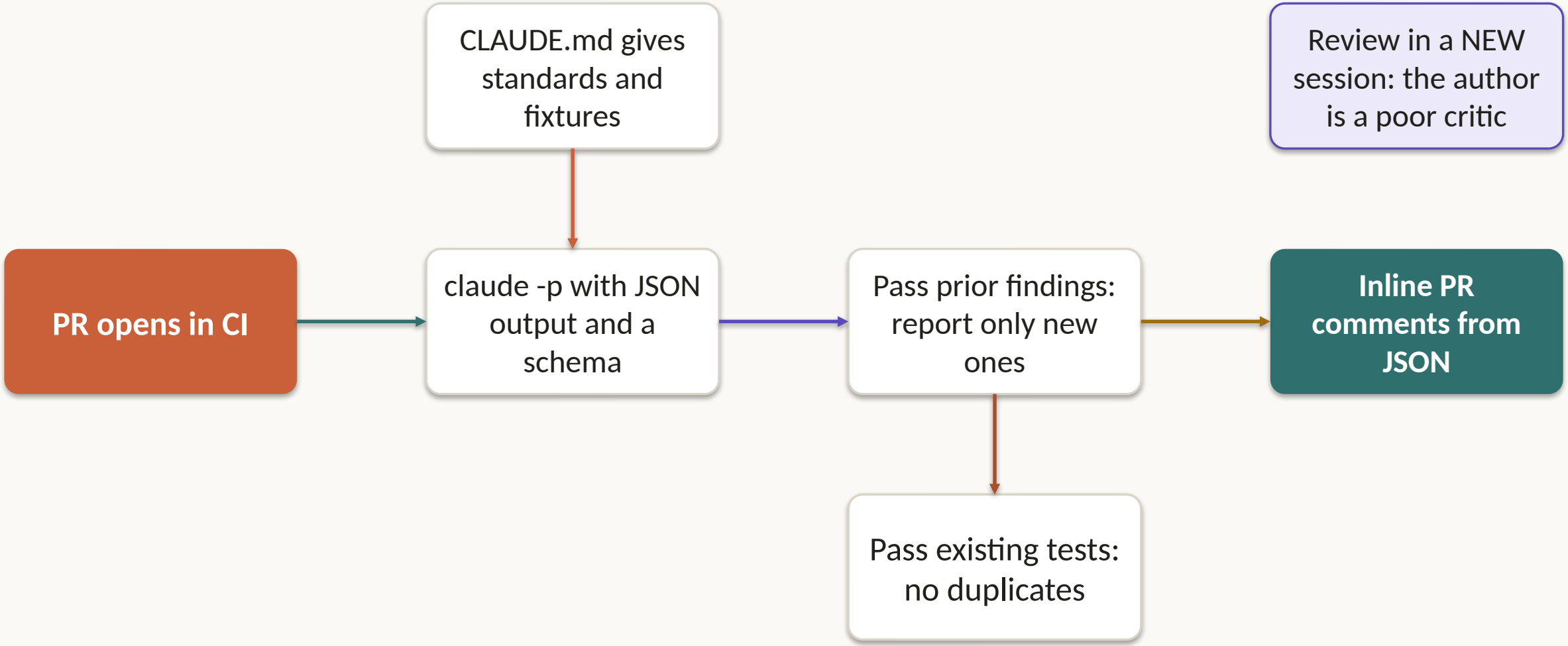
Browser: Simulator 3, Lab 35 · Claude Code: /lab 35 · Terminal, in simulator-3-multi-agent-research:

```
$ uv run research_system.py --case UC-27
```


© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

CI has no keyboard, so run claude -p

Without print mode the job waits for an answer nobody will type, until the runner dies.



© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Four flags turn Claude Code into a CI step

Structured, read-only, unattended.

The CI job, line by line

```
# per-file pass, from prompts/claude-command.txt in Simulator 5
claude -p "CI review of PR 418, per-file pass for $FILE. Apply prompts/review-criteria.txt." \
  --output-format json --json-schema "$(cat finding.schema.json)" \
  --permission-mode dontAsk --allowedTools Read Grep Glob --max-turns 6
```

-p

No keyboard needed, so no hung runner

--json-schema

Findings arrive validated, ready for inline PR comments

dontAsk + read-only tools

The job can look but cannot edit

© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

The nightly job hangs because nobody can press Enter

✗ Launch claude as if a developer were watching.

→

✓ Run claude -p.

→

One JSON envelope on stdout, and the process exits well inside the job timeout.

Simulator 5
Claude Code for Continuous Integration

Browser: Simulator 5, Lab 36 · Claude Code: /lab 36 · Terminal, in simulator-5-ci-cd:

```
$ uv run casekit.py --case UC-18
```


© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

A shared slash command still needs -p in CI

✗ Add a wrapper or a longer timeout.

→

✓ -p with the review prompt supplied non-interactively.

→

/pr-check runs unattended only because the job calls it through claude -p.

Simulator 5
Claude Code for Continuous Integration

Browser: Simulator 5, Lab 37 · Claude Code: /lab 37 · Terminal, in simulator-5-ci-cd:

```
$ uv run casekit.py --case UC-38
```


© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

TOPIC 4 · EXAM DOMAIN 4 · 20% OF THE EXAM

Prompt Engineering & Structured Output

Criteria, examples, schemas, validators: the path from a nice answer to a reliable one.



© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

TOPIC 4 MAP

6 task statements, 13 labs, 5 simulators

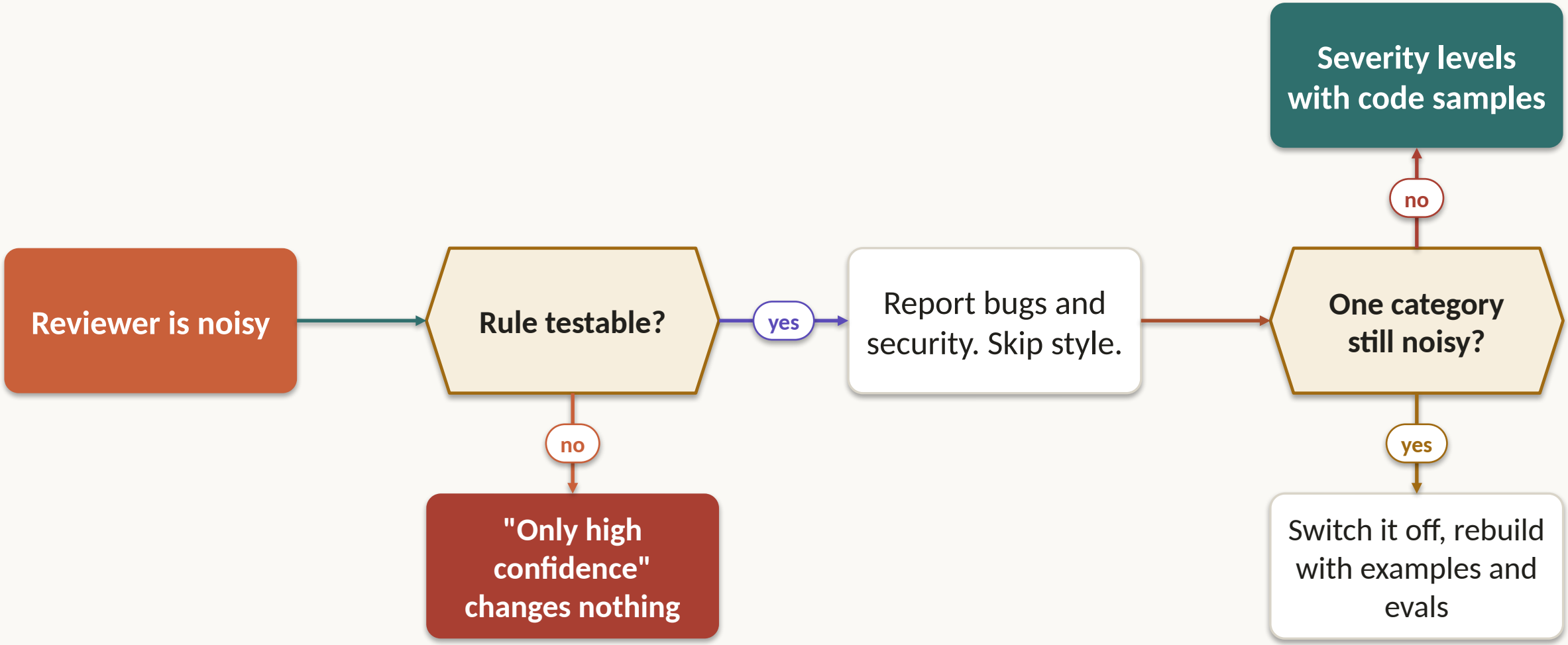
Concept first, then its labs, straight away.

Task statement	Sim 1	Sim 2	Sim 3	Sim 4	Sim 5	Sim 6
4.1 Design prompts with explicit criteria to improve precision and reduce false positives			39		38	
4.2 Apply few-shot prompting to improve output consistency and quality		40				
4.3 Enforce structured output using tool use and JSON schemas			41			42
4.4 Implement validation, retry, and feedback loops for extraction quality	43					44
4.5 Design efficient batch processing strategies	45,46					47
4.6 Design multi-instance and multi-pass review architectures			48,49		50	

© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

"Be conservative" is not a criterion

Name what to report and what to skip. One noisy category poisons trust in the rest.



© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

"Only high confidence" didn't quiet the reviewer

✗ A confidence instruction on top of a vague prompt.

→

✓ Testable report and skip rules: bugs and security in, style out.

→

Vague prompt: comments on names. Criteria: the None dereference in getPrice, nothing on naming.

Simulator 5
Claude Code for Continuous Integration

Browser: Simulator 5, Lab 38 · Claude Code: /lab 38 · Terminal, in simulator-5-ci-cd:

```
$ uv run casekit.py --case UC-39
```


© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Switch off the one noisy category, keep the accurate ones

✗ Disable the whole quality gate, or add "be careful".

→

✓ Omit the noisy category, then rebuild it with examples and evals.

→

You write the revised category list and an eval plan with a measurable threshold.

Simulator 3
Multi-Agent Research System

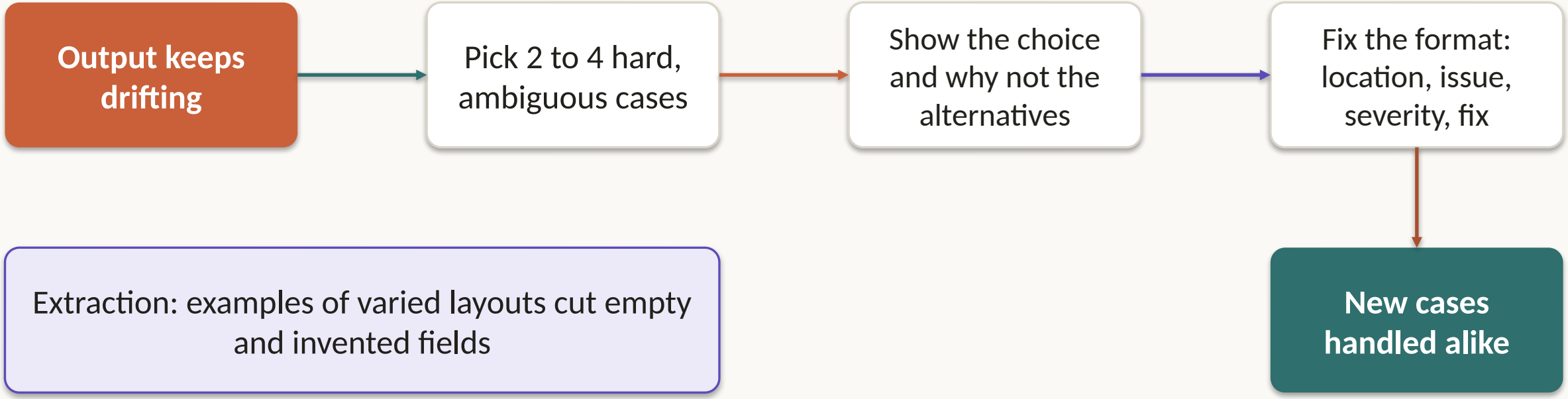
Browser: Simulator 3, Lab 39 · Claude Code: /lab 39 · Terminal, in simulator-3-multi-agent-research:

```
$ uv run research_system.py --case UC-20
```


© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Two to four examples draw the line prose can't

Show the ambiguous case and the reasoning. Claude generalises to cases you never listed.



© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

"Clean this up" means three things: show which one you mean

✗ More prose instructions in the /task skill.

→

✓ Two to four tagged examples with reasoning against the alternatives.

→

The three ambiguous requests classify as investigate, plan and modify.

Simulator 2
Code Generation with Claude Code

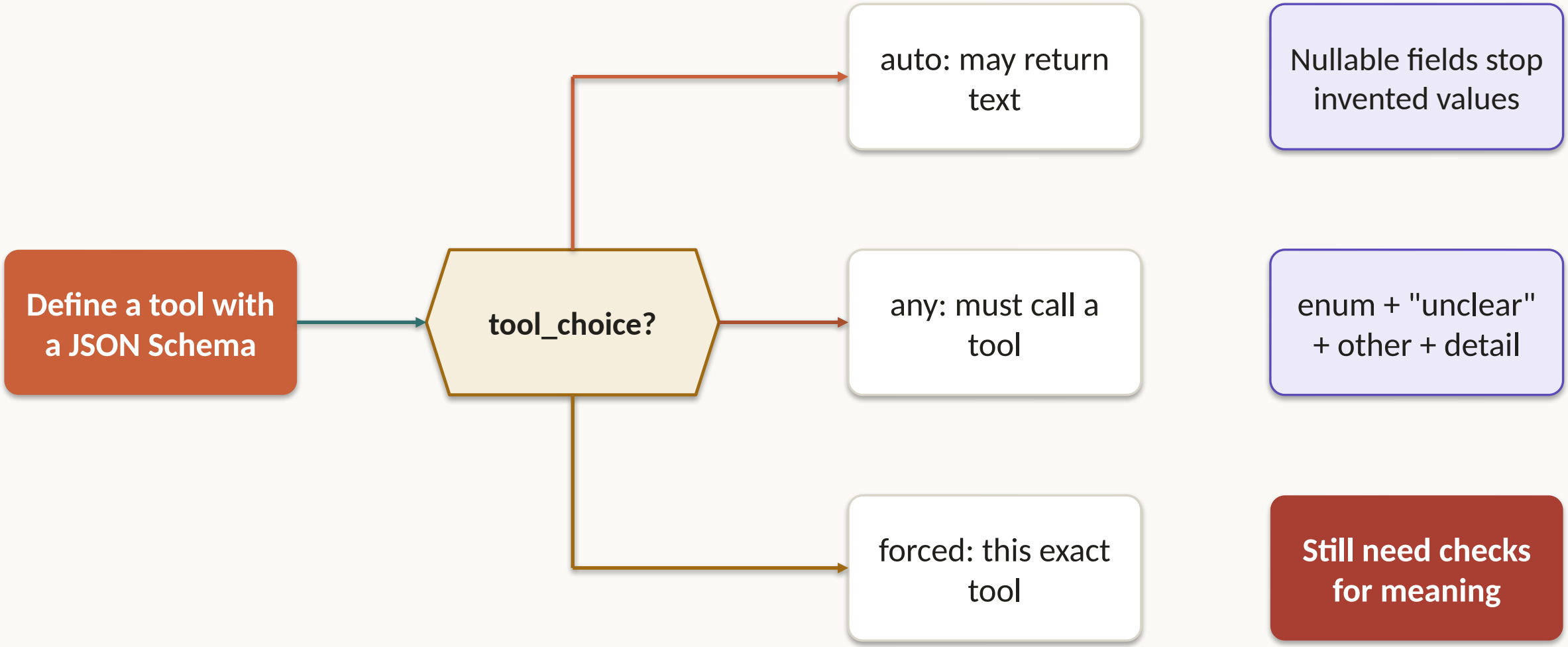
Browser: Simulator 2, Lab 40 · Claude Code: /lab 40 · Terminal, in simulator-2-code-generation:

```
$ uv run casekit.py --case UC-23
```


© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

The schema guarantees shape, never meaning

Strict tool use ends JSON syntax errors. Totals that don't add up still get through.



© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Valid JSON, wrong refund

Why strict schemas still need a validator behind them.

The schema is happy

Every required field present

All numbers are numbers

`stated_total`: 38.00

Store credit in `shipping_refund`

Your validator is not

Line items sum to 35.00, so `conflict_detected`

Store credit never counts as cash

Refund must equal its parts

Failures go back to Claude, by name

© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Vague passages forced into a category they don't fit

✗ A closed enum with no way out.

→

✓ Add unclear and other, plus an other_detail field.

→

Every value stays in the enum. Vague passages use unclear. Any other carries other_detail.

Simulator 3
Multi-Agent Research System

Browser: Simulator 3, Lab 41 · Claude Code: /lab 41 · Terminal, in simulator-3-multi-agent-research:

```
$ uv run research_system.py --case UC-28
```


© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Try tool_choice three ways and read the failures

✗ Assume any forced setting works on every model.

→

✓ Know the options, and test against your model.

→

The Agent SDK has no tool_choice setting, so the script asks for it in the prompt. Read what Claude did, and the lesson in the evidence.

Simulator 6
Structured Data Extraction

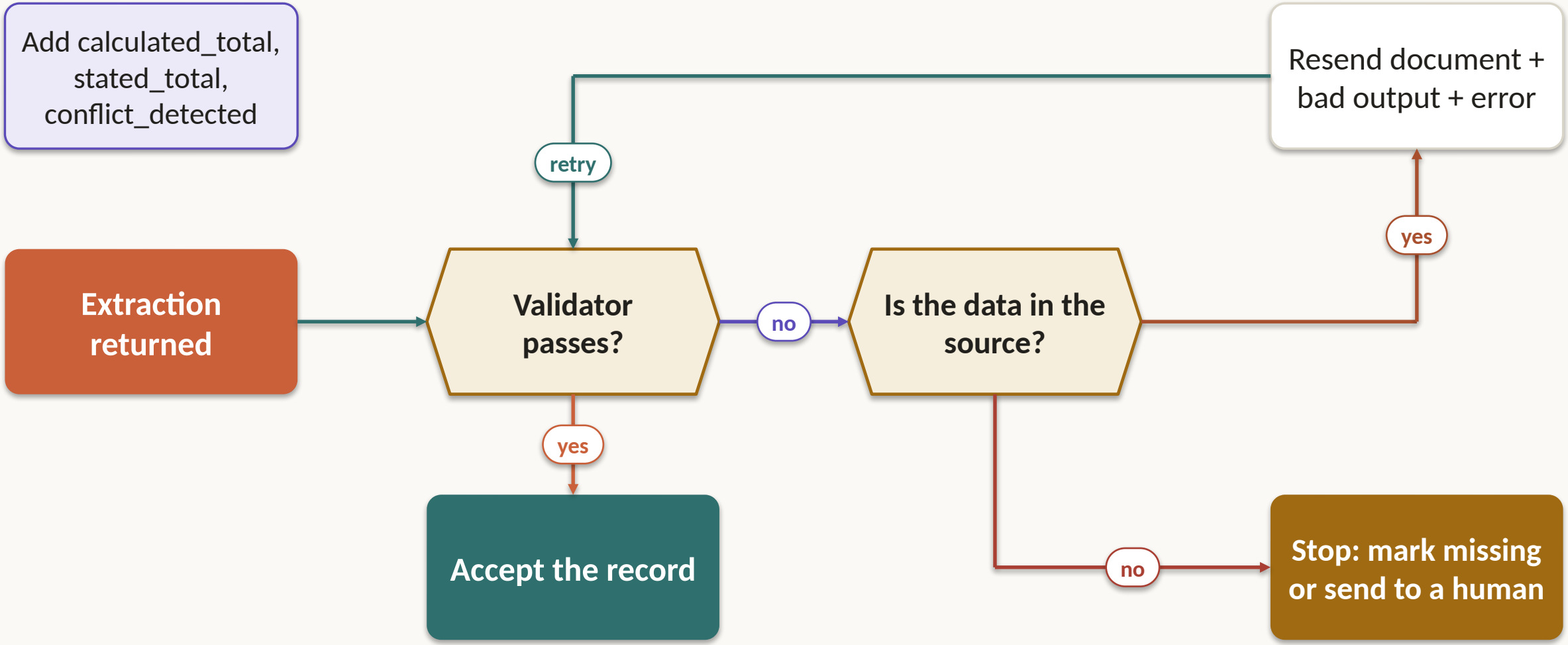
Browser: Simulator 6, Lab 42 · Claude Code: /lab 42 · Terminal, in simulator-6-structured-extraction:

```
$ uv run extract.py --tool-choice any --max-docs 2
$ uv run extract.py --tool-choice forced --max-docs 2
```


© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Retry with the error, or stop if the data isn't there

No prompt can recover a schedule that was never attached.



© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Valid JSON, wrong refund: add business rules

✗ More required fields, or drop the schema.

→

✓ Validate invariants before execution and feed failures back.

→

Cash refund is $105. The $8 shipping stays store credit.

Simulator 1
Customer Support Resolution Agent

Browser: Simulator 1, Lab 43 · Claude Code: /lab 43 · Terminal, in simulator-1-customer-support:

```
$ uv run backend_tools.py --demo
$ uv run support_agent.py --case CS-13
$ uv run support_agent.py --case CS-13 --no-validation --tag novalidation
```


© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Retry when the format is wrong, stop when the data is absent

✗ Retry everything until something comes back.

→

✓ Resend the document, bad output and error. Stop if the source lacks it.

→

DOC-03 keeps both totals and goes to a human. DOC-07 names what is missing and goes to a human without a retry. A format error gets a retry.

Simulator 6
Structured Data Extraction

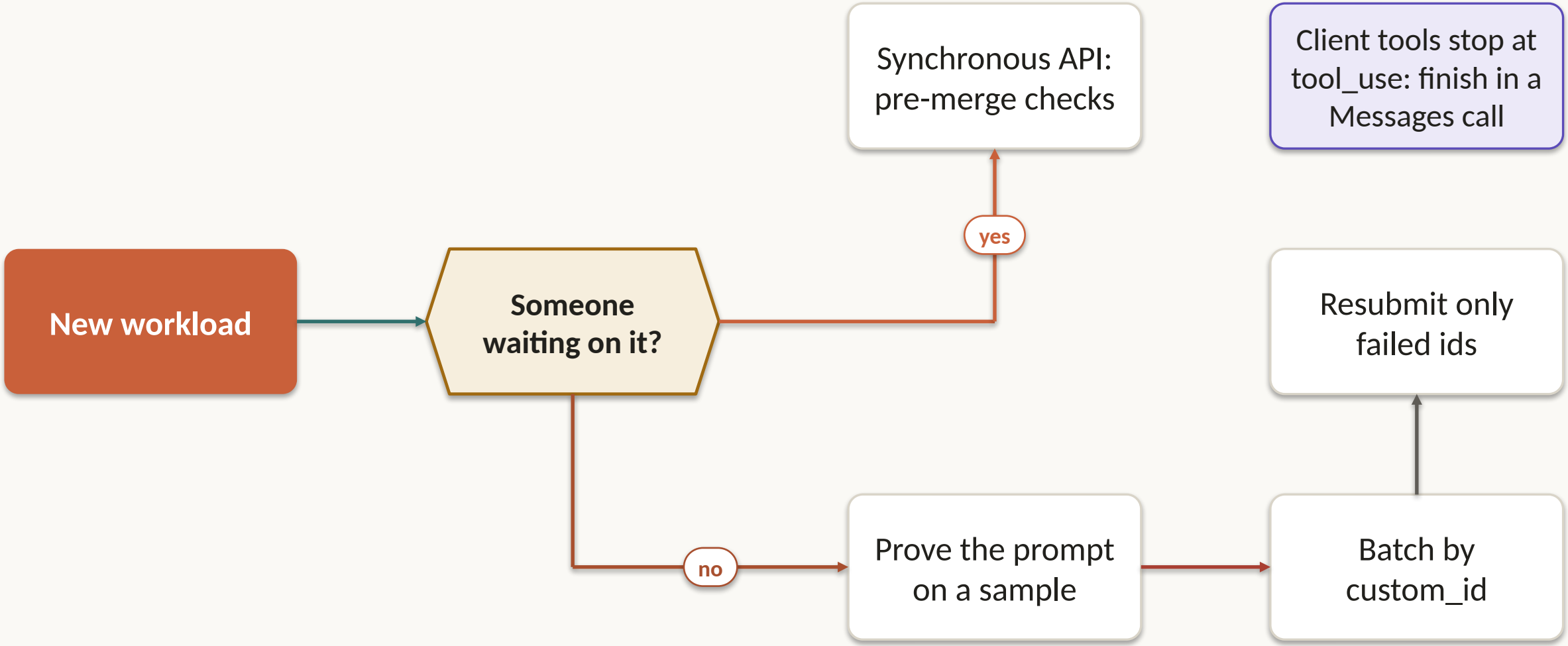
Browser: Simulator 6, Lab 44 · Claude Code: /lab 44 · Terminal, in simulator-6-structured-extraction:

```
$ uv run extract.py
```


© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Half the price, up to 24 hours, no tool loop

Message Batches suit work nobody is waiting for. Prove the prompt before you scale it.



© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Prove the template on ten, then run thirty-five thousand

Batches are cheap per token and expensive to redo.



35,000

transcripts wait until the pilot passes

50%

cheaper, up to 24 h, no SLA

Workload	API
Blocking pre-merge check	Synchronous Messages
Interactive upload validation	Synchronous Messages
Overnight archive or weekly audit	Message Batches
Case needing a client tool	Batch, then finish in Messages

© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Prove the template before 35,000 transcripts

✗ Run the whole archive tonight, fix bad rows tomorrow.

→

✓ Evaluation set, refined template, rerun, then batch.

→

Weak template: not ready. Real template: ready for the overnight batch.

Simulator 1
Customer Support Resolution Agent

Browser: Simulator 1, Lab 45 · Claude Code: /lab 45 · Terminal, in simulator-1-customer-support:

```
$ uv run triage.py --pilot --template prompts/triage_template_v0.md
$ uv run triage.py --pilot
```


© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Client tools can't run inside a batch

✗ Expect the batch to execute MCP tools mid-request.

→

✓ Batch the analysis, run returned tool calls, continue in Messages.

→

Rows needing a tool stop at stop_reason: tool_use.

Simulator 1
Customer Support Resolution Agent

Browser: Simulator 1, Lab 46 · Claude Code: /lab 46 · Terminal, in simulator-1-customer-support:

```
$ uv run triage.py --batch
```


© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Nightly job on Batches, upload flow stays synchronous

✗ Move everything to Batches to save money.

→

✓ Route by who is waiting.

→

interactive_upload routes to sync, nightly_archive to batch, with a 0.5 cost multiplier.

Simulator 6
Structured Data Extraction

Browser: Simulator 6, Lab 47 · Claude Code: /lab 47 · Terminal, in simulator-6-structured-extraction:

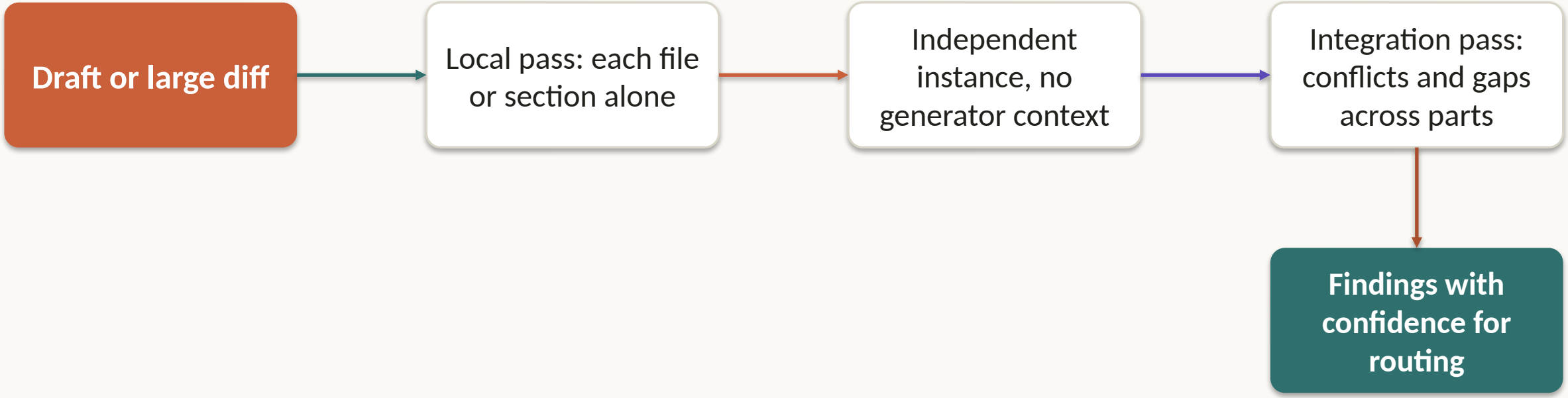
```
$ uv run extract.py --case UC-32
```


© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

The author is the worst reviewer of their own draft

Fresh eyes and focused passes catch what self-review and longer thinking miss.

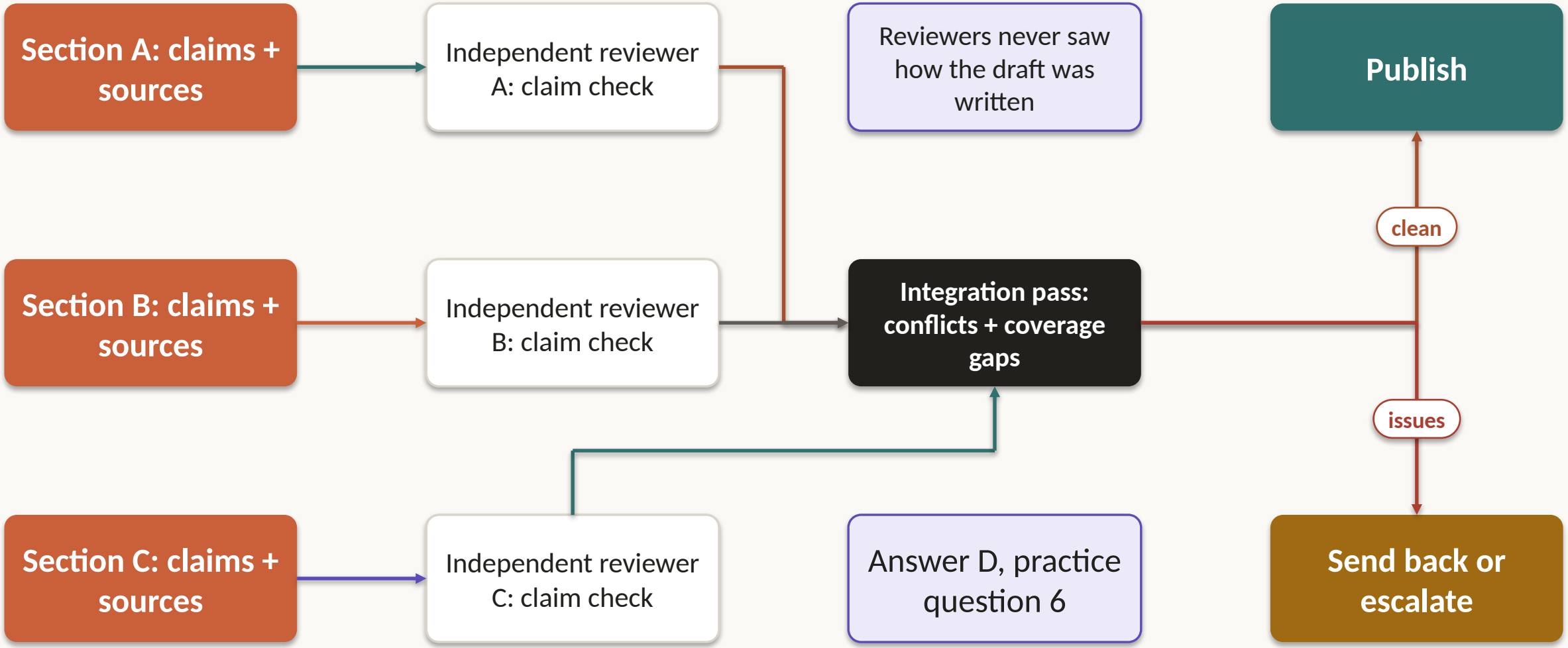
Self-review in the same session keeps its own reasoning



© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Independent eyes on each section, one pass across them all

Reviewers catch claims. The integration pass catches contradictions.



© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

The author reviewing its own draft misses its own errors

✗ The report generator checks its own work.

→

✓ Independent reviewers on claim-source bundles, then an integration pass.

→

Two or more reviewers launch in one response, and a separate integration pass follows.

Simulator 3
Multi-Agent Research System

Browser: Simulator 3, Lab 48 · Claude Code: /lab 48 · Terminal, in simulator-3-multi-agent-research:

```
$ uv run research_system.py --case UC-06
```


© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Source-level passes first, integration second

✗ One long prompt reviews everything at once.

→

✓ An explicit prompt chain with checkpoints you can log and score.

→

Reviewers get bundles smaller than the synthesis, and integration runs last.

Simulator 3
Multi-Agent Research System

Browser: Simulator 3, Lab 49 · Claude Code: /lab 49 · Terminal, in simulator-3-multi-agent-research:

```
$ uv run research_system.py --case UC-19
```


© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

14 files, shallow comments: split the review

✗ A bigger context window or a stricter prompt.

→

✓ Per-file local passes, then one cross-file integration pass.

→

Only the integration pass finds the jobs.py call with the old signature.

Simulator 5
Claude Code for Continuous Integration

Browser: Simulator 5, Lab 50 · Claude Code: /lab 50 · Terminal, in simulator-5-ci-cd:

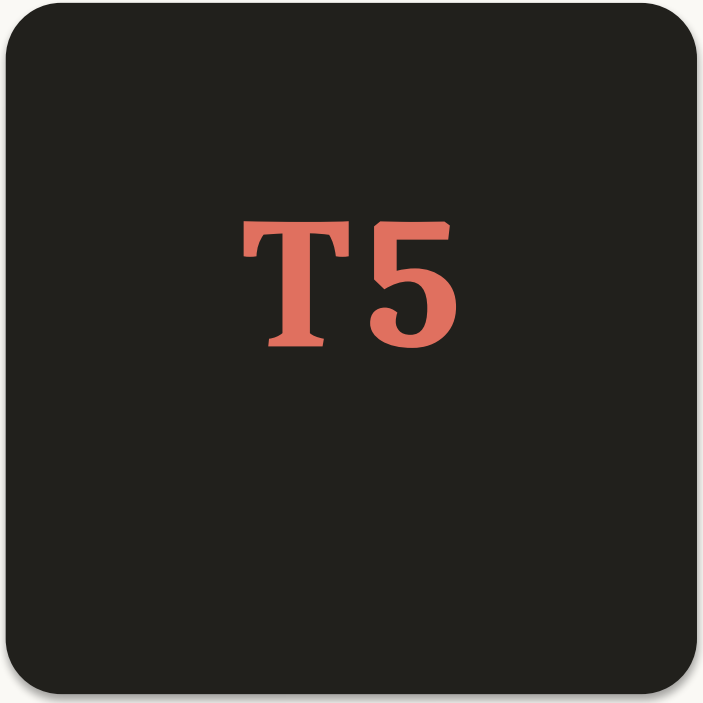
```
$ uv run ci_review.py; echo "exit $?"
```


© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

TOPIC 5 · EXAM DOMAIN 5 · 15% OF THE EXAM

Context Management & Reliability

Long sessions forget. Multi-agent runs lie by omission. Here's how to stop both.



© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

TOPIC 5 MAP

6 task statements, 11 labs, 6 simulators

Concept first, then its labs, straight away.

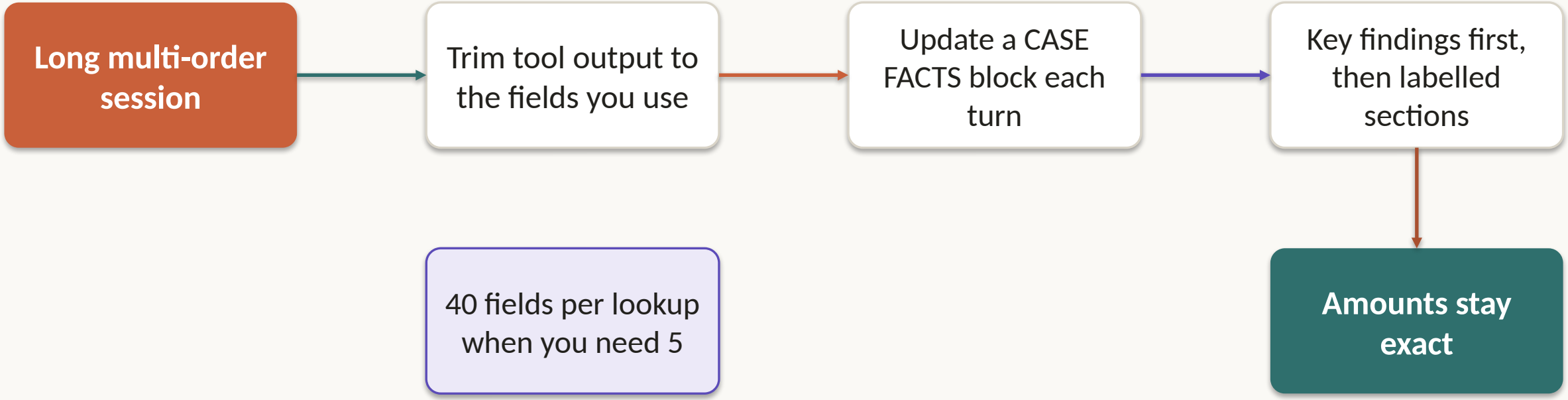
Task statement	Sim 1	Sim 2	Sim 3	Sim 4	Sim 5	Sim 6
5.1 Manage conversation context to preserve critical information across long interactions	51		53		52	
5.2 Design effective escalation and ambiguity resolution patterns	54					
5.3 Implement error propagation strategies across multi-agent systems			55			
5.4 Manage context effectively in large codebase exploration				56,58		57
5.5 Design human review workflows and confidence calibration		59				60
5.6 Preserve information provenance and handle uncertainty in multi-source synthesis			61			

© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Facts live outside the summary

"About $45" is how a refund lands on the wrong order.

Progressive summary rounds numbers and dates away



© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Three orders, one refund, zero mix-ups

The block that keeps amounts exact when history gets summarised.

Every reply starts with this block

```
CASE FACTS
Customer : C-1001 (Mei Lin Tan)
ORD-5503 : $45.00 duplicate charge, refunded
ORD-5501 : $120.00 validated, not issued
ORD-5504 : shipped, ETA 9 November 2026

# Rebuilt every turn. Never summarised.
```

40+

fields per order lookup when the agent needs about 5: trim before they pile up

1

refund on the right order after three turns across three orders

Lost in the middle

Long inputs get read at the start and the end. Put key findings first, then give every detailed section an explicit header.

© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Refund amount jumped to the wrong order

✗ Let older turns get summarised.

→

✓ A case-facts block updated and shown on every turn.

→

After three turns the only refund is $45 on ORD-5503.

Simulator 1
Customer Support Resolution Agent

Browser: Simulator 1, Lab 51 · Claude Code: /lab 51 · Terminal, in simulator-1-customer-support:

```
$ uv run support_agent.py --case CS-12
```


© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

40-field tool results push the review to its limit

✗ A bigger model, or summarise later.

✓ Wrap the tool so it returns only review-relevant fields.

Verbose payload: tens of kilobytes. Compact payload: a few hundred bytes.

Simulator 5
Claude Code for Continuous Integration

Browser: Simulator 5, Lab 52 · Claude Code: /lab 52 · Terminal, in simulator-5-ci-cd:

```
$ uv run casekit.py --case UC-13
```


© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Taxonomy drift: write it to a state file

✗ Resend the growing history and hope.

→

✓ A structured research-state file loaded into later prompts.

→

research_state.json holds taxonomy and source map, and the synthesis packet quotes it.

Simulator 3
Multi-Agent Research System

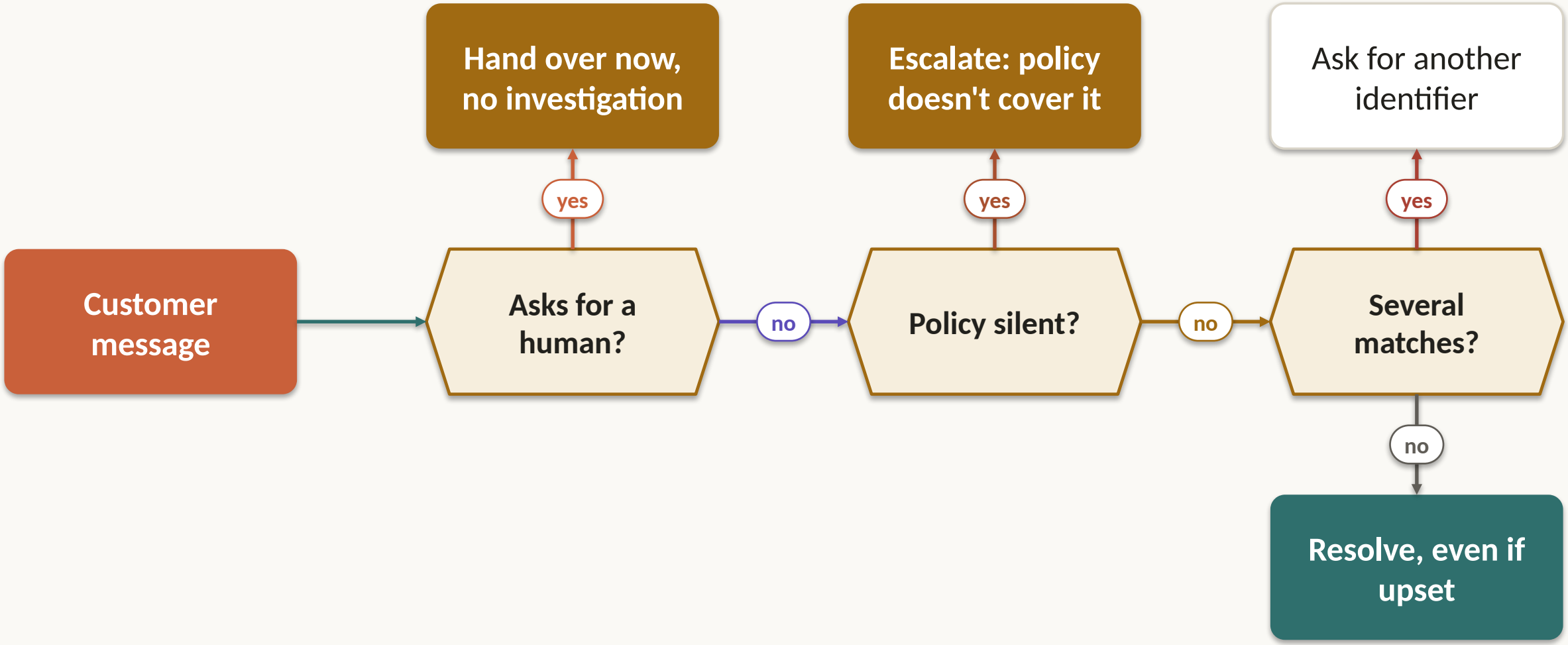
Browser: Simulator 3, Lab 53 · Claude Code: /lab 53 · Terminal, in simulator-3-multi-agent-research:

```
$ uv run research_system.py --case UC-21
```


© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Escalate on triggers, not on tone

Anger and a confidence score predict nothing. Three triggers do.



© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Three triggers. Three things that only feel like triggers.

Calibrate here and first-contact resolution climbs.

These three trigger escalation

Customer asks for a human: hand over now

Policy silent: competitor price match

No progress possible

These three do not

Angry tone: acknowledge, then resolve

Model says "low confidence"

Several account matches: ask for another id

© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

55% first-contact resolution: calibrate with criteria and examples

✗ Escalate on sentiment or a confidence score.

→

✓ Explicit escalation criteria with few-shot examples.

→

Human request and price match escalate. Two names: it asks. Angry but routine: resolved.

Simulator 1
Customer Support Resolution Agent

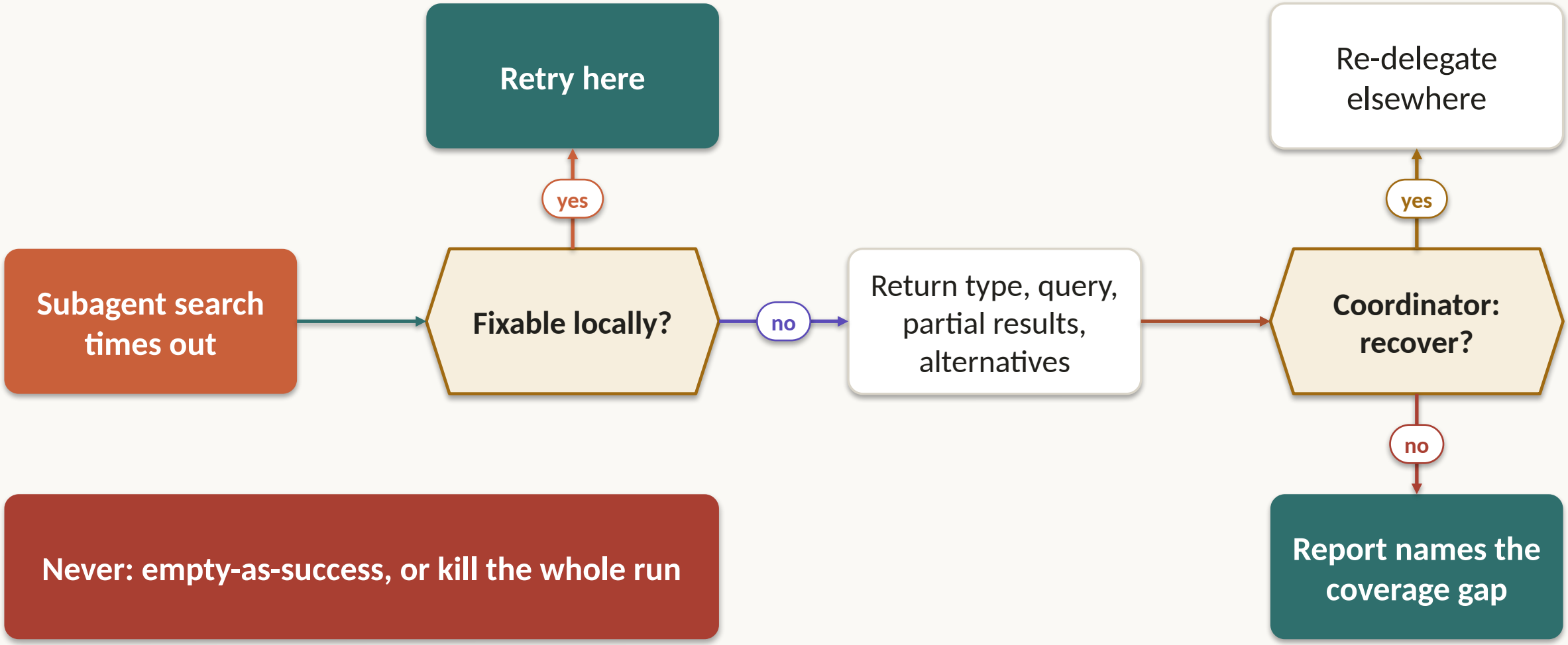
Browser: Simulator 1, Lab 54 · Claude Code: /lab 54 · Terminal, in simulator-1-customer-support:

```
$ uv run support_agent.py --case CS-05 --case CS-08 --case CS-09 --case CS-10
$ uv run evaluate.py
```


© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Say what failed, what you tried, what you still have

A blank result pretending to be success is the most expensive bug in a pipeline.



© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

The search timed out: tell the coordinator everything

✗ "Search unavailable", or empty results marked as success.

→

✓ Failure type, attempted query, partial results, alternatives.

→

The run finishes, and the report names visual arts as a coverage gap.

Simulator 3
Multi-Agent Research System

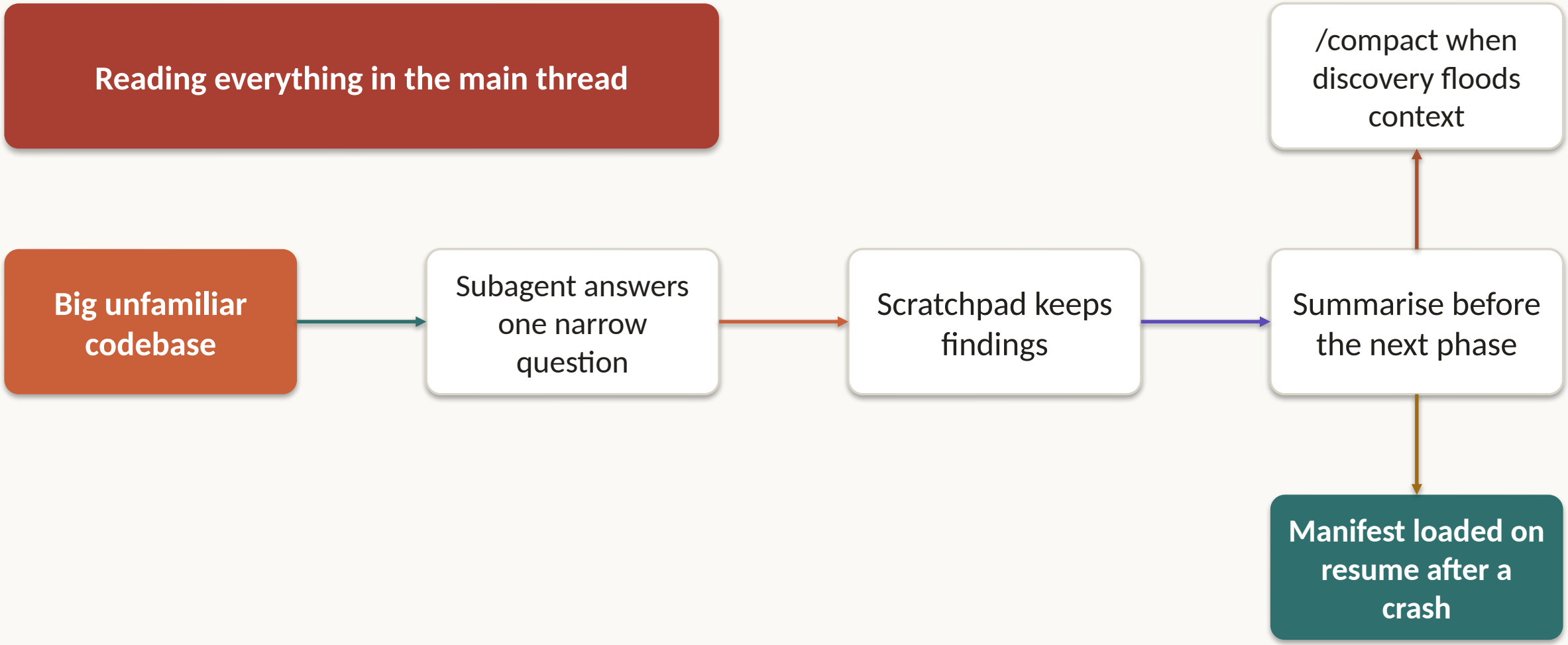
Browser: Simulator 3, Lab 55 · Claude Code: /lab 55 · Terminal, in simulator-3-multi-agent-research:

```
$ uv run research_system.py
```


© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Delegate the noise, keep the signal

When answers drift to "typical patterns", your context has already degraded.



© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Delegate the noisy search, keep a scratchpad

✗ Read everything in the main thread.

→

✓ An explore subagent, findings in NOTES.md.

→

An explore subagent runs, and NOTES.md names calculate_total and report.py.

Simulator 4
Developer Productivity with Claude

Browser: Simulator 4, Lab 56 · Claude Code: /lab 56 · Terminal, in simulator-4-developer-productivity:

```
$ uv run explorer.py --case UC-D3
```


© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Keep discovery out of the main thread

✗ Paste dozens of modules into one conversation.

→

✓ Explore subagent with explicit scope, implement from its summary.

→

Prints the delegation prompt to paste into Claude Code. The main thread stays short.

Simulator 6
Structured Data Extraction

Browser: Simulator 6, Lab 57 · Claude Code: /lab 57 · Terminal, in simulator-6-structured-extraction:

```
$ uv run extract.py --case UC-22
```


© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Crash recovery from a manifest

✗ Start exploring again from nothing.

→

✓ Export state to a manifest and load it on resume.

→

Fresh session id, no Grep, no subagent, and the tests pass.

Simulator 4
Developer Productivity with Claude

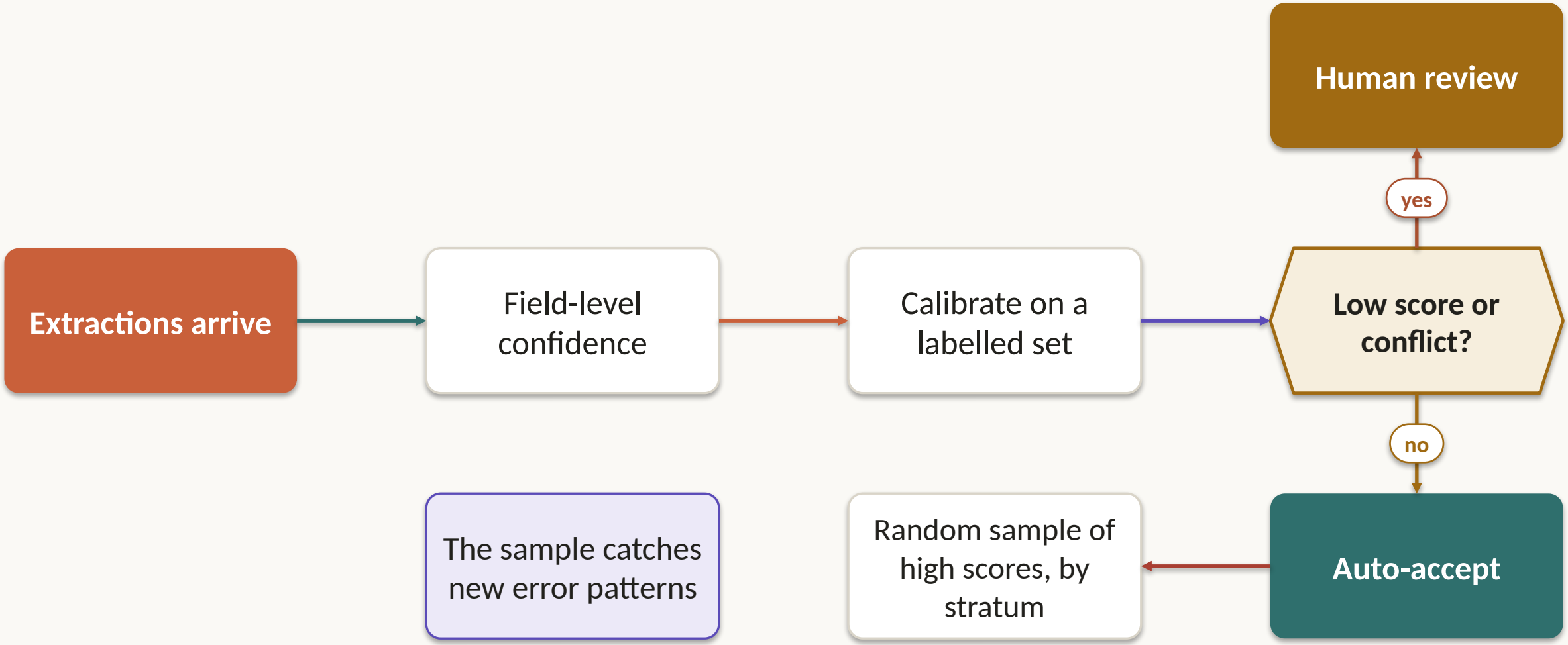
Browser: Simulator 4, Lab 58 · Claude Code: /lab 58 · Terminal, in simulator-4-developer-productivity:

```
$ uv run explorer.py --case UC-D5
```


© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

97% overall can hide the one field that fails

Slice accuracy by document type and field before you remove a single reviewer.



© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Acceptance looks great. Two fields don't.

✗ Cut human review because the overall rate is high.

→

✓ Field-level confidence, calibrated on labelled examples.

→

One confidence per field. Under 0.8 goes to review, as do signatures under 0.95.

Simulator 2
Code Generation with Claude Code

Browser: Simulator 2, Lab 59 · Claude Code: /lab 59 · Terminal, in simulator-2-code-generation:

```
$ uv run casekit.py --case UC-33
```


© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

The aggregate masks a segment: break it and watch

✗ One overall accuracy number.

→

✓ Accuracy by document type and by field.

→

Change one invoice total: only by_document_type.invoice falls.

Simulator 6
Structured Data Extraction

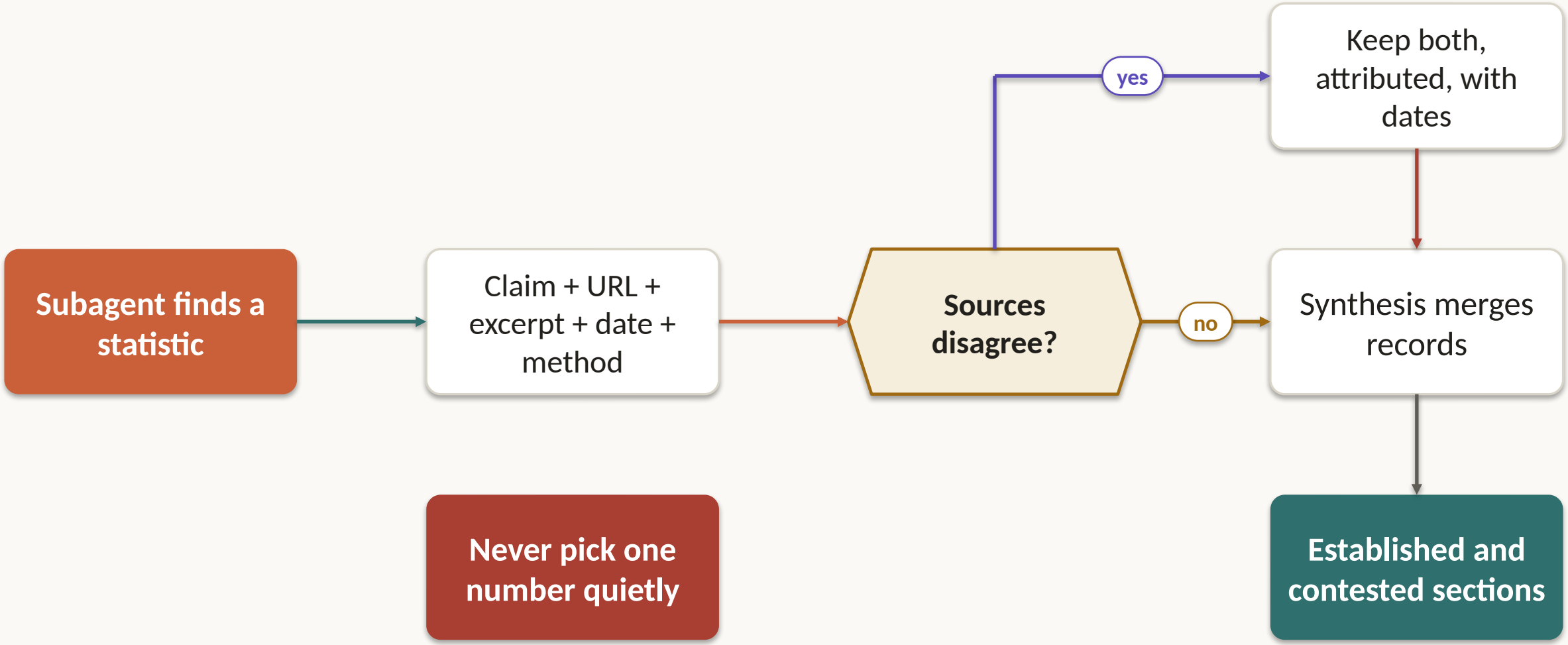
Browser: Simulator 6, Lab 60 · Claude Code: /lab 60 · Terminal, in simulator-6-structured-extraction:

```
$ uv run extract.py
$ uv run python -m json.tool evidence/accuracy.json
```


© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

The source travels with the claim, all the way to the report

Summaries drop attribution first. Structured claim records can't lose it.



© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Conflicts stay visible, with names attached

A coordinator can only reconcile what synthesis kept.

A claim record from Simulator 3's mock sources

```
{"claim": "41% in 2025, up from 23% in 2023",
 "document": "WIPO-Creative-Economy-2025.pdf",
 "page": 14,
 "published": "2025-11-30",
 "methodology_note": "not stated in excerpt"}
```

Well established

Agreement across sources, cited

Contested

41% (WIPO) next to 55% (UNESCO)

Coverage gaps

Visual arts search timed out, and says so

Different dates are not a contradiction

Require the publication date on every statistic, so a 2021 figure and a 2026 figure read as change over time. Tables for numbers, prose for news.

© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

A vendor benchmark isn't a peer-reviewed study

✗ A summary plus a general source list.

→

✓ Claim records with location, date, excerpt and method.

→

Each finding carries document, page, date and method note. 41% and 55% stay side by side.

Simulator 3
Multi-Agent Research System

Browser: Simulator 3, Lab 61 · Claude Code: /lab 61 · Terminal, in simulator-3-multi-agent-research:

```
$ uv run research_system.py --case UC-24
```


© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

WHAT YOU ACHIEVED

You ran the six systems the exam asks about

Five domains, each one proved on a simulator rather than recited.

Agents that stop for the right reason

stop_reason loops, coordinators, Task, hooks, sessions

Tools Claude picks correctly

Descriptions, structured errors, scoped access, MCP

Claude Code your team shares

CLAUDE.md, skills, path rules, plan mode, claude -p

Output you can trust

Criteria, few-shot, schemas, validators, batches

Systems that survive long runs

Case facts, escalation, provenance, calibration

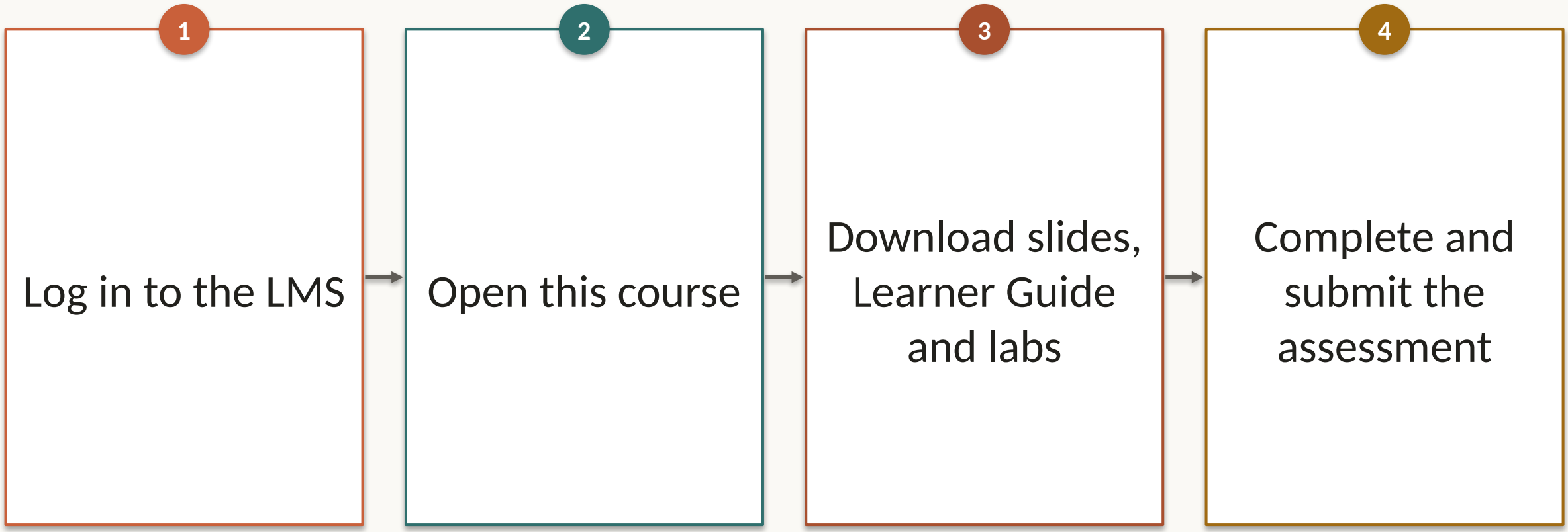
Evidence for every claim

61 labs, each with a file you can open and check

© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Courseware & Assessment on the LMS

lms-tms.tertiaryinfotech.com holds everything. Submit your assessment there too.



© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

TEST YOURSELF

Practice Exam

Attempt it timed. Then rerun the lab behind anything you missed.

https://exams.tertiaryinfotech.com/practice-exams/anthropic/anthropic-cca-foundations

1. Sit it under timed conditions
2. Read every explanation
3. Find the lab for each miss
4. Retake the exam

© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Assessment

Two papers, one hour each, open book.

WA

Written Assessment (SAQ): 10 questions, 60 minutes

CS

Case Study: 5 tasks, one lab screenshot each, 60 minutes

LMS

Submit your completed answers there

Before you start

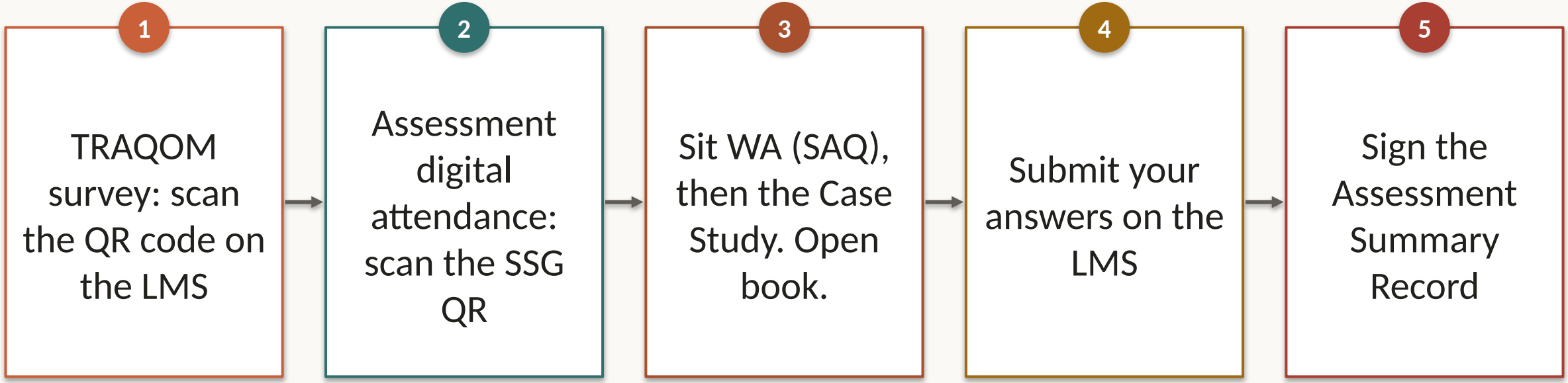
Open book: slides, Learner Guide and approved materials only.
Complete the TRAQOM survey, then the Assessment digital attendance.
Each Case Study task needs one screenshot from a lab you run yourself.
An appeal process is available if you need one.

© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

ON ASSESSMENT DAY

Assessment Flow

Five steps, in this order, every time.



TRAQOM QR code and answer submission both sit on the LMS: lms-tms.tertiaryinfotech.com

© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Digital Attendance (Mandatory)

Three scans a day keep your funding eligibility intact.

1. Trainer displays the SSG QR code
2. Scan with your phone camera
3. Submit your attendance
4. Repeat: AM, PM and Assessment

75%

minimum attendance for assessment and funding

Mandatory for WSQ-funded courses

Take the AM, PM and Assessment digital attendance. Miss a scan, and tell your trainer the same day.

© Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Thank You

Build systems that stay useful when tools fail, context grows and confidence runs out.

Tertiary Infotech Academy Pte Ltd
UEN: 201200696W

© Tertiary Infotech Academy Pte Ltd · UEN 201200696W